

Embedded System Design

UNIT 2

Anuj Kumar Sharma
Assistant Professor
Dept. of ECE
FET, GKV
Haridwar

Index

- Discussion of Syllabus of unit 2
- Second lab Experiment
- **Custom Single Purpose Processors.**
- Optimizing program.
- FSMD, Data path & FSM.
- General purpose processors
- ASIP's (Application Specific Instruction set Programming) Processors
- DSP Chips

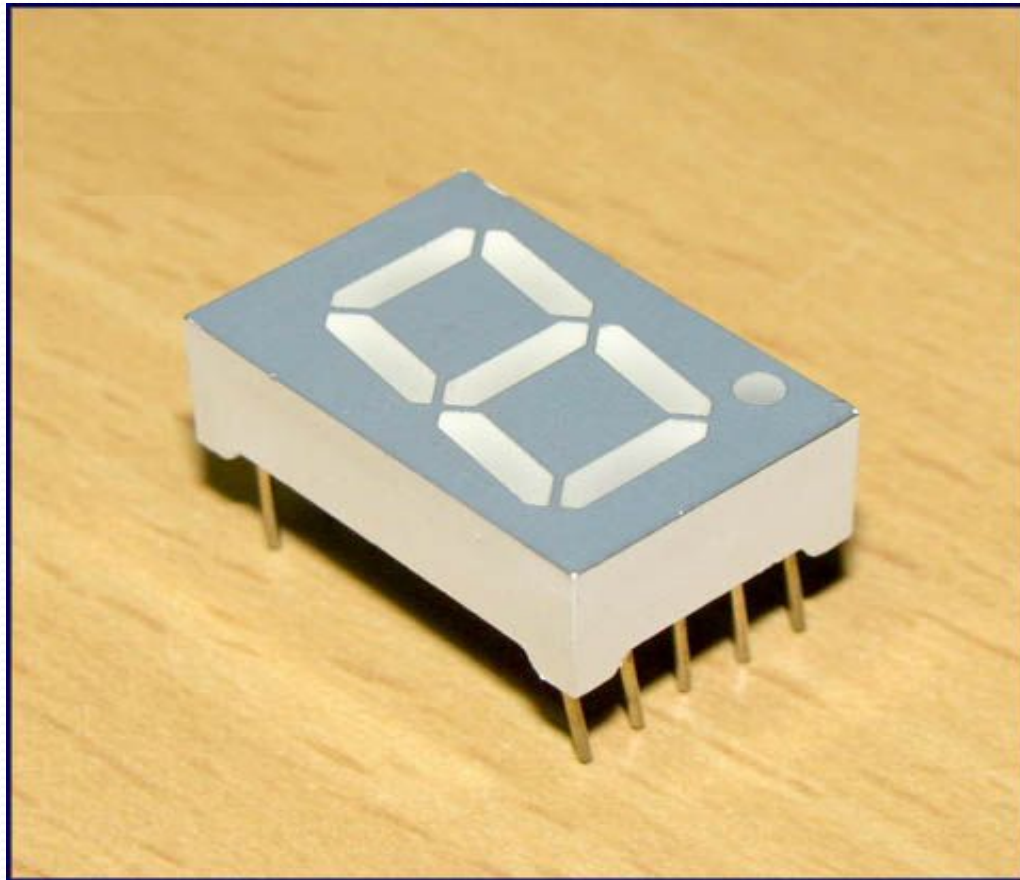
Discussion of Syllabus of unit 2

- [embedded system design syllabus.docx](#)
- [Document 60.pdf](#)

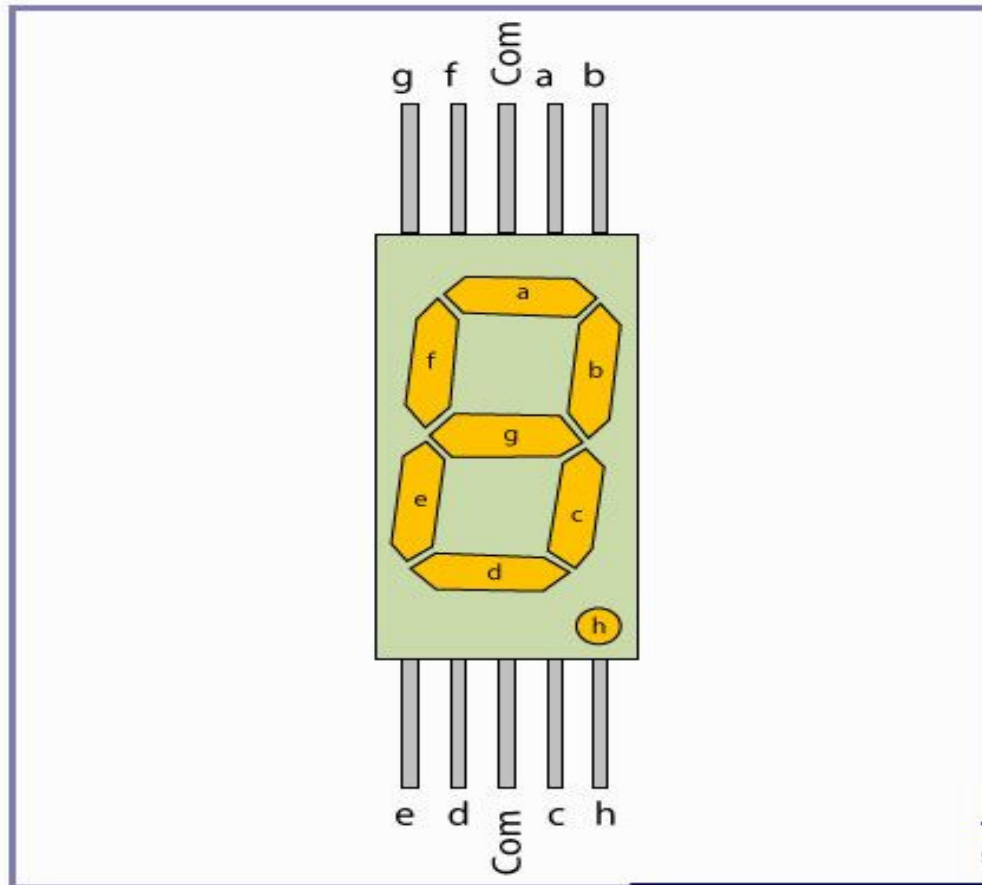
Second Lab Experiment

- Object: To interface the Seven segment Display with microcontroller (89C51, 8051) and implement it on Breadboard and proteus
- Component required : Seven segment display , microcontroller , crystal oscillator , resistors, capacitors , power supplies.

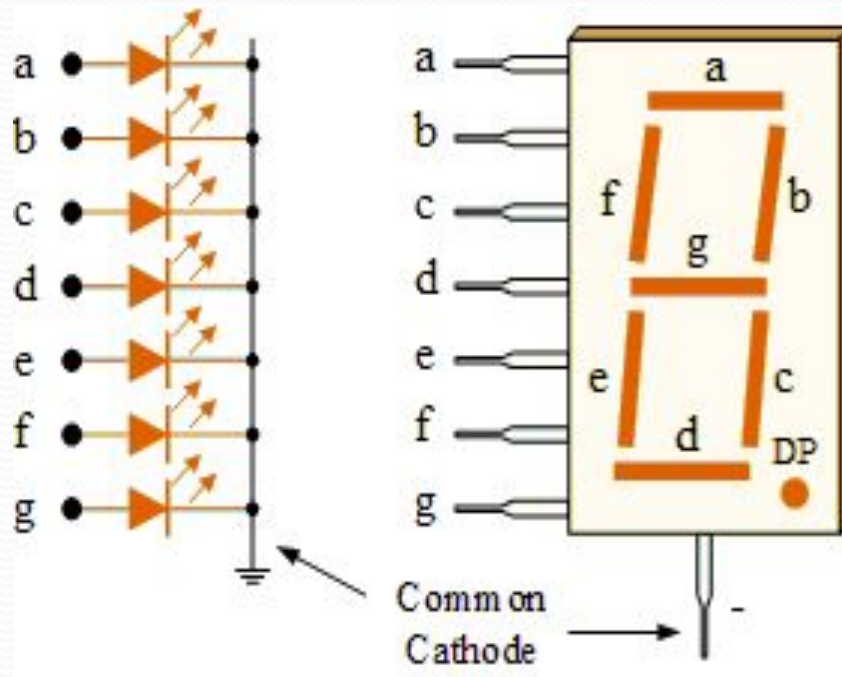
Conti...



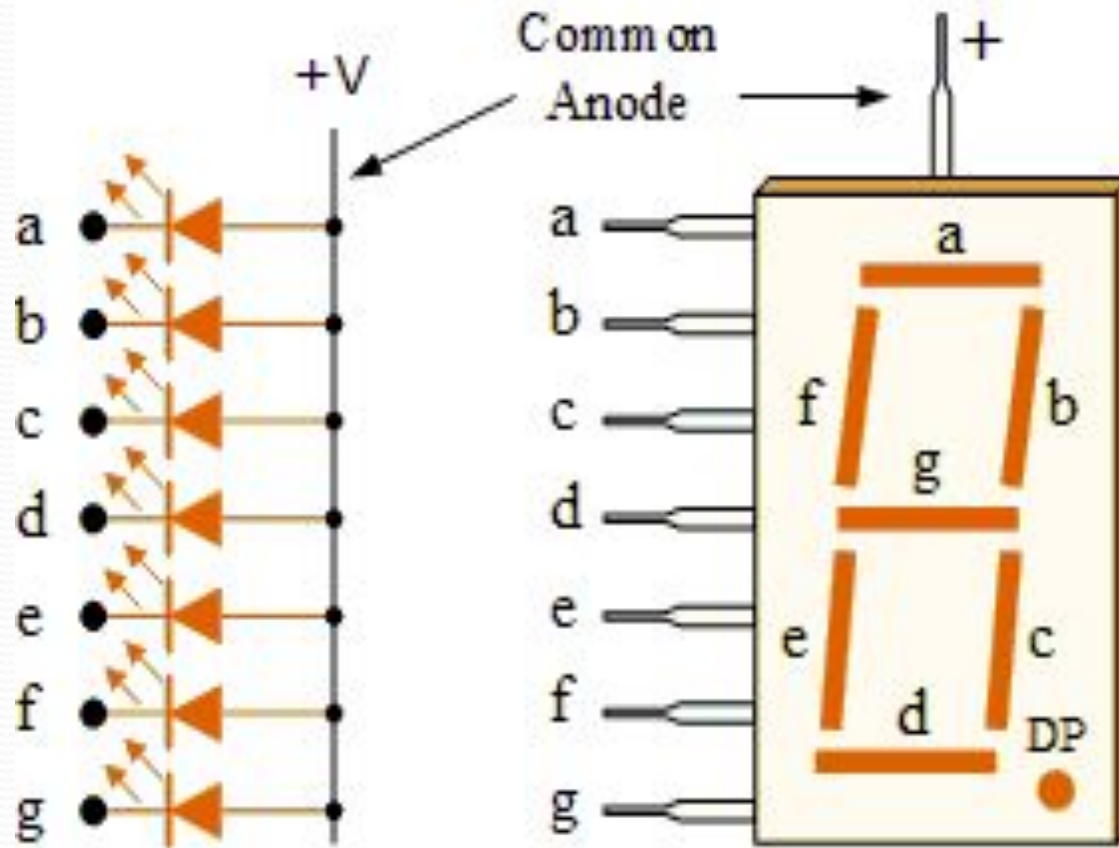
Conti...

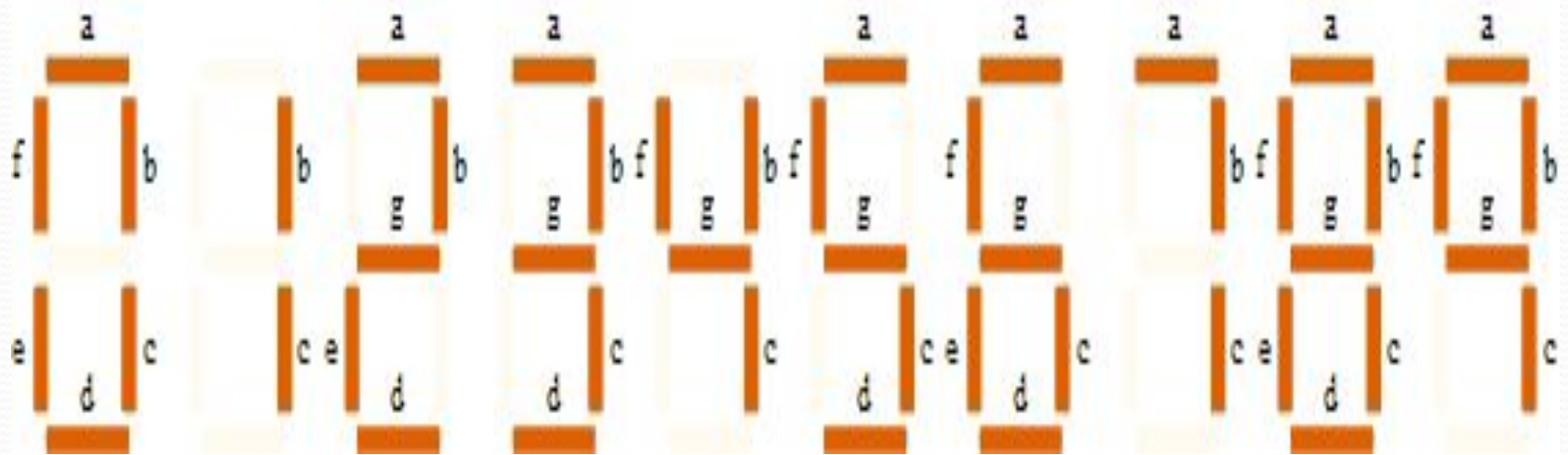


Common cathode



Common anode





Truth table

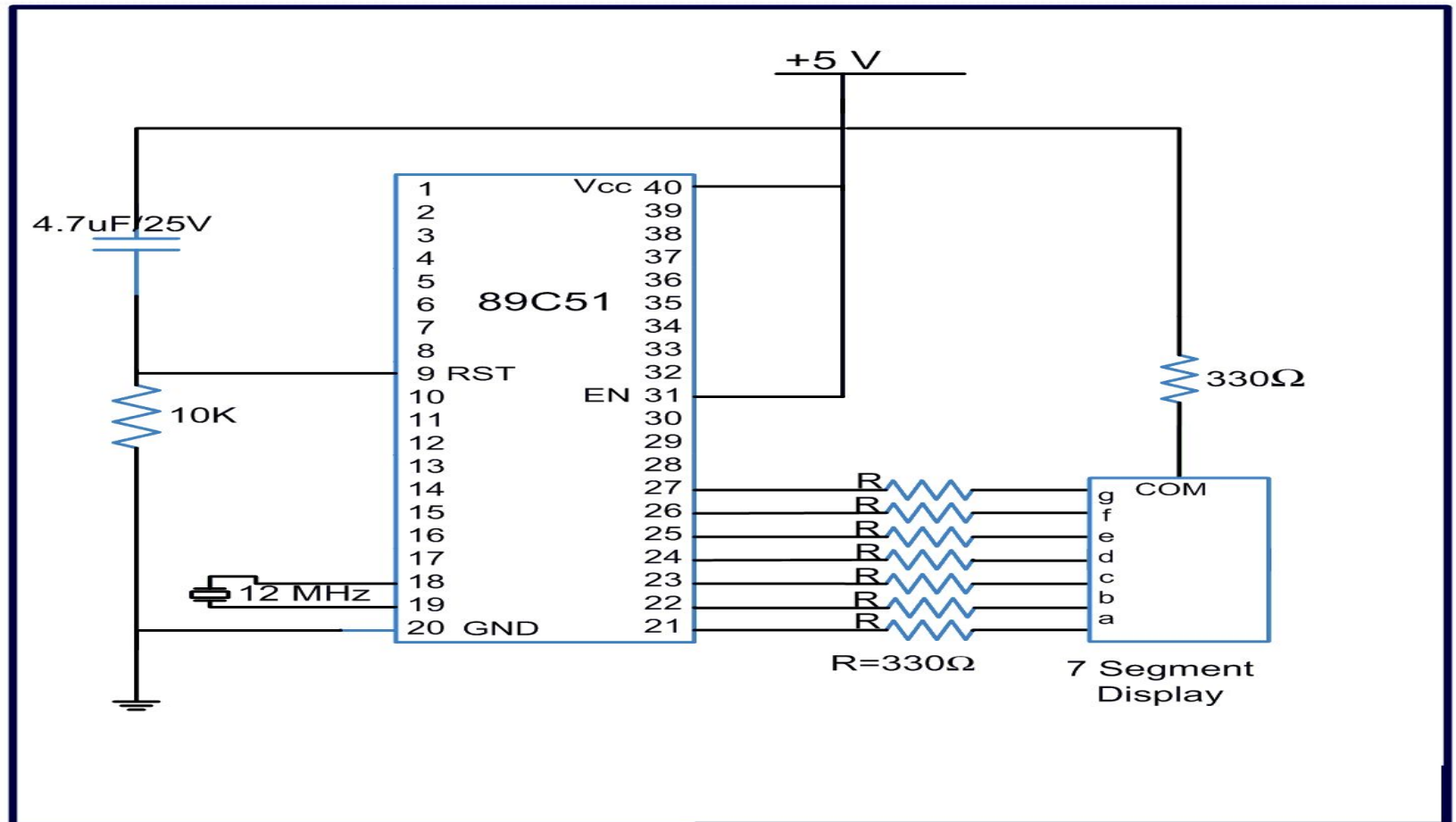
Numbers	Common Cathode		Common Anode	
	(DP)GFEDCBA	HEX Code	(DP)GFEDCBA	HEX Code
0	00111111	0x3F	11000000	0xC0
1	00000110	0x06	11111001	0xF9
2	01011011	0x5B	10100100	0xA4
3	01001111	0x4F	10110000	0xB0
4	01100110	0x66	10011001	0x99
5	01101101	0x6D	10010010	0x92
6	011111101	0x7D	10000010	0x82
7	00000111	0x07	11111000	0xF8
8	01111111	0x7F	10000000	0x80
9	01101111	0x6F	10010000	0x90

Digit	Display	gfedcba	abcdefg	a	b	c	d	e	f	g
0		0x3F	0x7E	on	on	on	on	on	on	off
1		0x06	0x30	off	on	on	off	off	off	off
2		0x5B	0x6D	on	on	off	on	on	off	on
3		0x4F	0x79	on	on	on	on	off	off	on
4		0x66	0x33	off	on	on	off	off	on	on
5		0x6D	0x5B	on	off	on	on	off	on	on
6		0x7D	0x5F	on	off	on	on	on	on	on
7		0x07	0x70	on	on	on	off	off	off	off
8		0x7F	0x7F	on	on	on	on	on	on	on
9		0x6F	0x7B	on	on	on	on	off	on	on
A		0x77	0x77	on	on	on	off	on	on	on
b		0x7C	0x1F	off	off	on	on	on	on	on
C		0x39	0x4E	on	off	off	on	on	on	off
d		0x5E	0x3D	off	on	on	on	on	off	on
E		0x79	0x4F	on	off	off	on	on	on	on
F		0x71	0x47	on	off	off	off	on	on	on

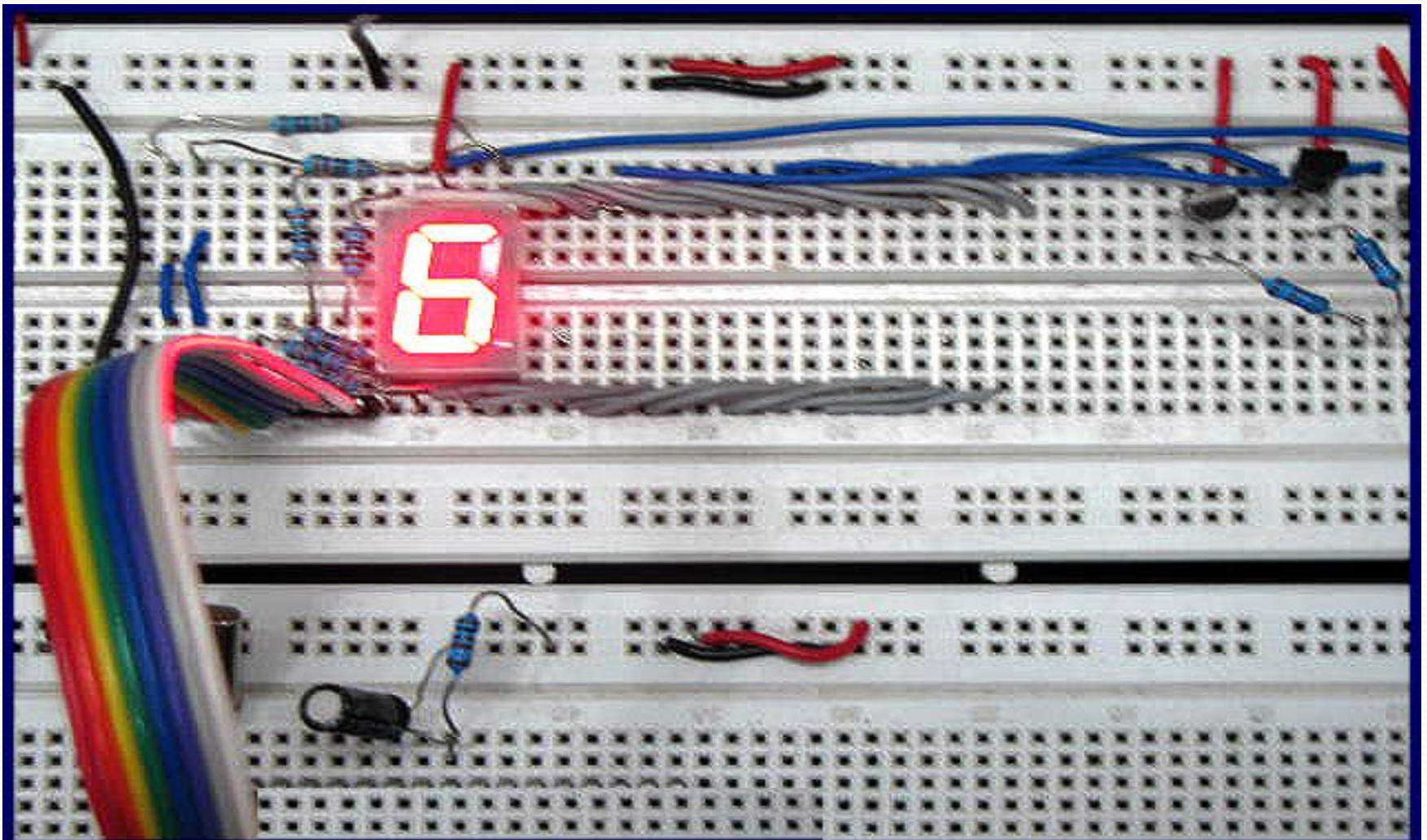
Conti...

- Theory :
- seven segment display Theory.docx

Conti....



Conti...



Coding

- [seven segment display coding.docx](#)

Custom Single Purpose Processor

[unit2.pdf](#)

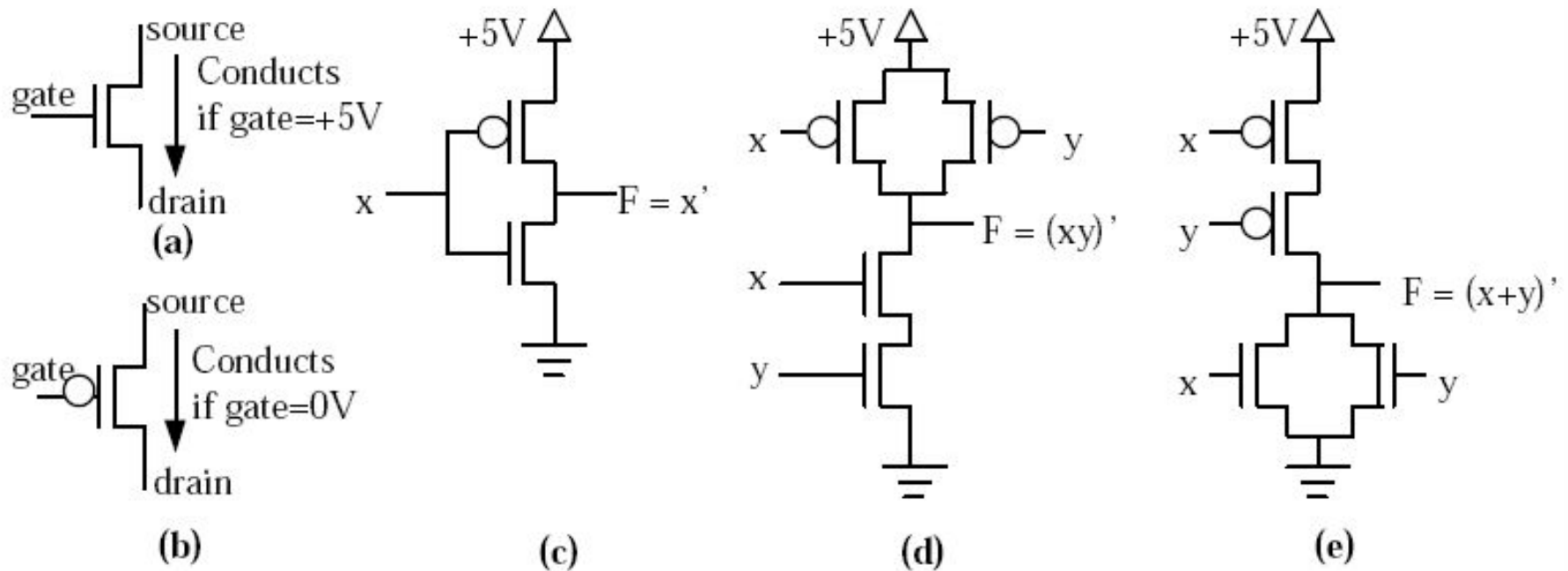
- A single-purpose processor is a digital system intended to solve a specific computation task.
- A custom single purpose processor to execute a specific task within the ES
- An embedded system designer choosing to use a custom single-purpose, rather than a general-purpose, processor to implement part of a system's functionality may achieve several benefits.
 - performance may be fast
 - size may be small
- Here its start with a review of combinational and sequential design, and then describe a method for converting programs to custom single-purpose processors.

Combinational logic design

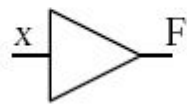
- A combinational circuit is a digital circuit whose output is purely a function of its current inputs; such a circuit has no memory of past inputs.
- A transistor is the basic electrical component of digital systems. Combinations of transistors form components called logic gates.
- The basic principle of a NPN transistor to act as a switch is, a high voltage (typically +5 Volts as logic 1) is applied to the gate, the transistor conducts, so current flows. When low voltage (refer to as logic 0, typically ground,) is applied to the gate, the transistor does not conduct.

Creation of Gates using transistors

CMOS transistor implementations of some basic logic gates: (a) nMOS transistor, (b) pMOS transistor, (c) inverter, (d) NAND gate, (e) NOR gate.



Basic logic gates



$F = x$
Driver

x	F
0	0
1	1



$F = x y$
AND

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1



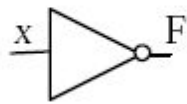
$F = x + y$
OR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



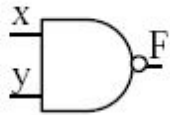
$F = x \oplus y$
XOR

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0



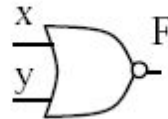
$F = x'$
Inverter

x	F
0	1
1	0



$F = (x y)'$
NAND

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0



$F = (x + y)'$
NOR

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0



$F = x \nabla y$
XNOR

x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

Combinational circuit design

- Q: y is 1 if a is equal to 1, or b and c is equal to 1.

z is 1 if a is equal to 1 and b is equal to 1 or if b or c is equal to 1, but not both.

(b) Truth table

Inputs			Outputs	
a	b	c	y	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

(c) Output equations

$$y = a'bc + ab'c' + ab'c + abc' + abc$$

$$z = a'b'c + a'bc' + ab'c + abc' + abc$$

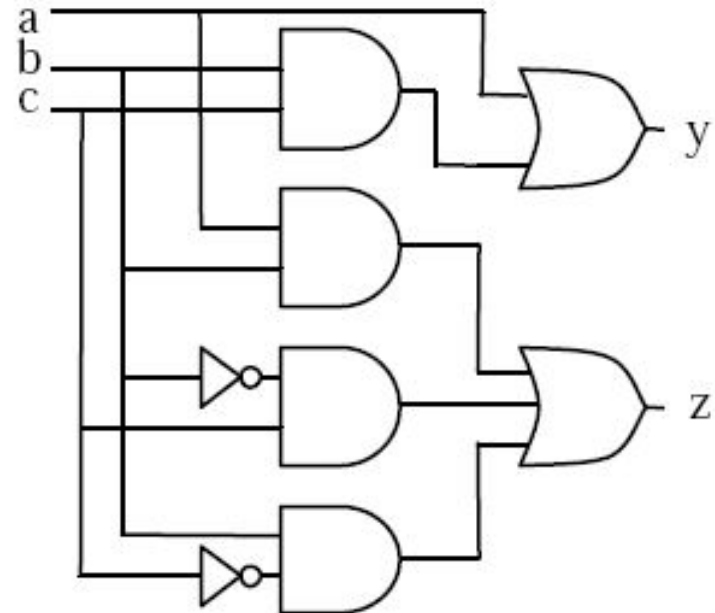
(d) Minimized output equations

y	a \ bc	00	01	11	10
	0	0	0	1	0
	1	1	1	1	1

$$y = a + bc$$

z	a \ bc	00	01	11	10
	0	0	1	0	1
	1	0	1	1	1

$$z = ab + b'c + bc'$$

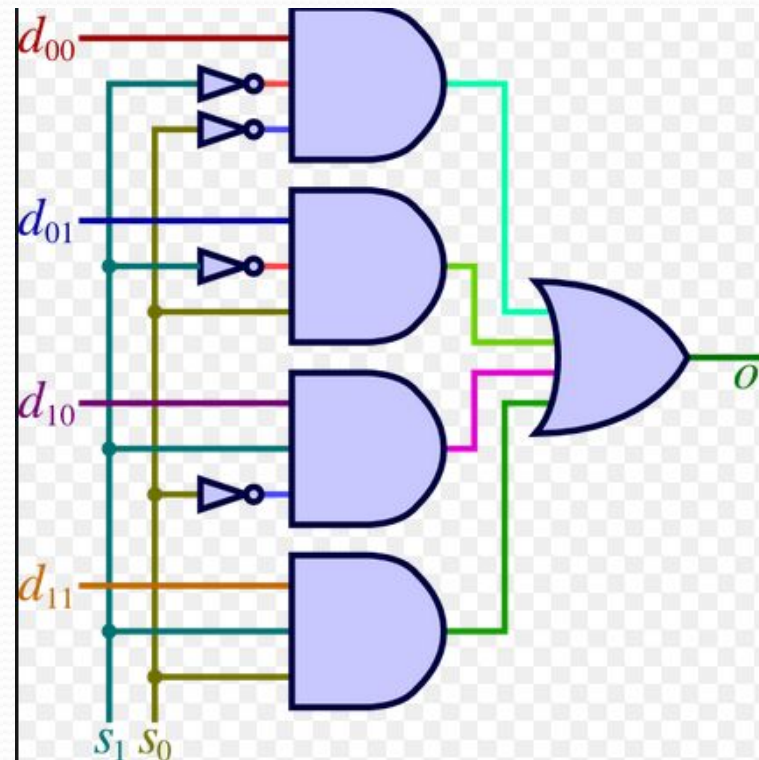
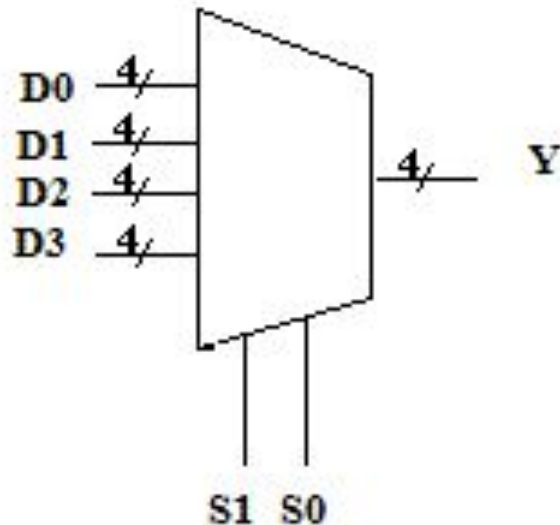


RT –level combinational components

- RT(Register Transfer) level uses combination components that are more power full than gates.
- Such Components are
 - Multiplexer
 - Decoder
 - Adder
 - Comparator
 - ALU

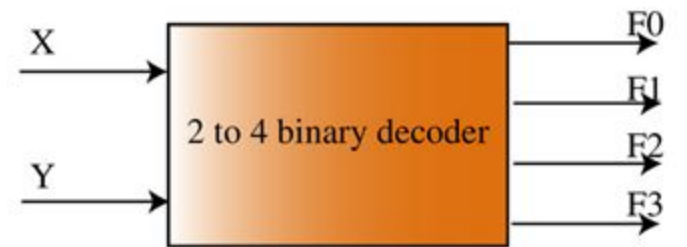
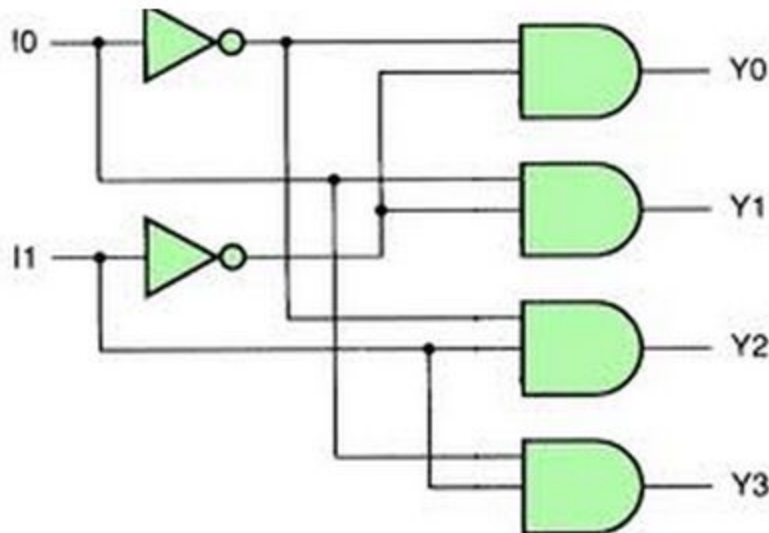
Multiplexer

- A multiplexor, sometimes called a selector, allows only one of its data inputs to pass through to the output according to the selection pins inputs.
- If there are m data inputs, then there are $\log_2(m)$ select lines.



Decoder

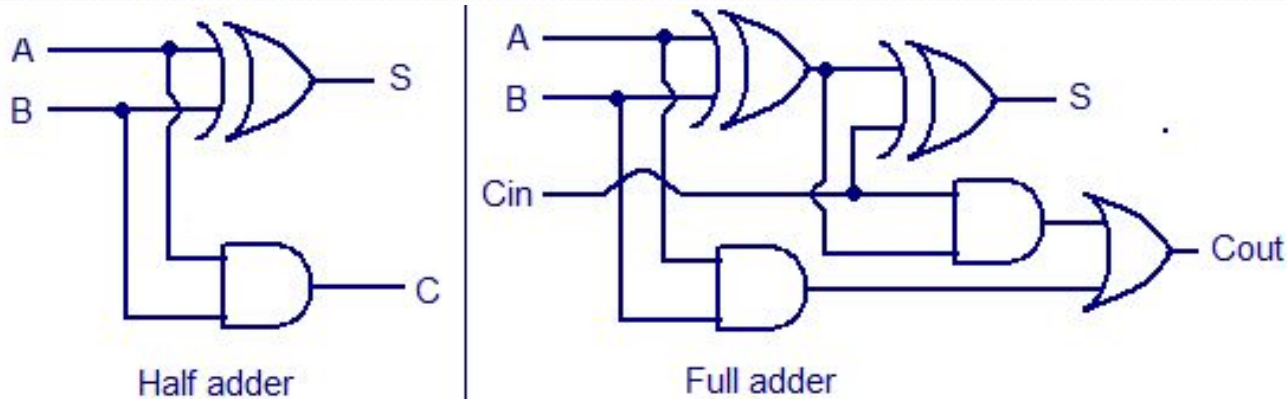
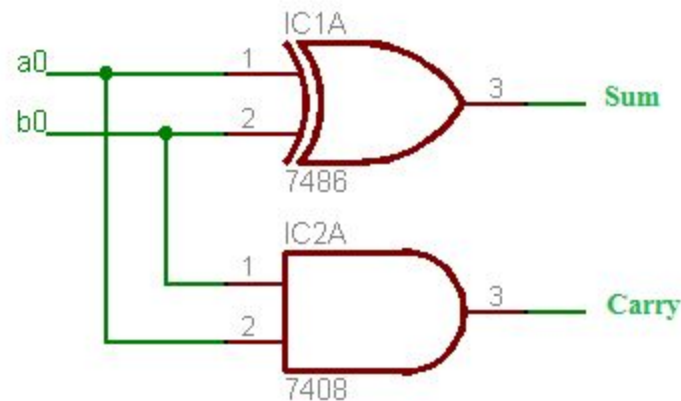
- A decoder converts its binary input I into a one-hot output O . "One-hot" means that exactly one of the output lines can be 1 at a given time.
- Thus, if there are n outputs, then there must be $\log_2(n)$ inputs.



X	Y	F0	F1	F2	F3
0	0	1	0	0	0
1	0	0	1	0	0
0	1	0	0	1	0
1	1	0	0	0	1

Adder

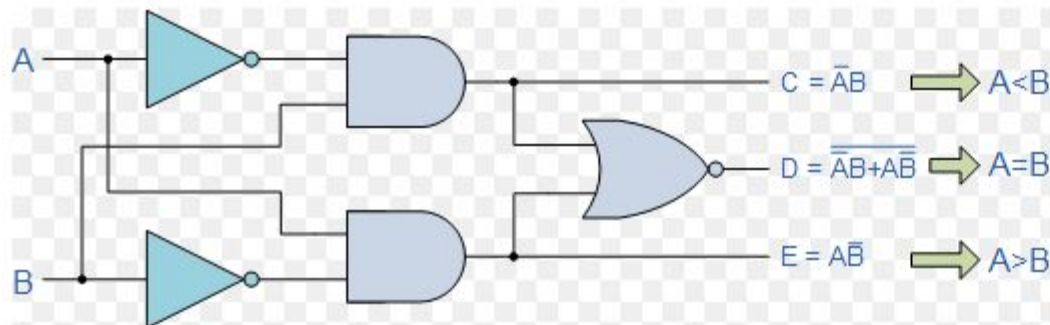
- An adder adds two n-bit binary inputs A and B, generating an n-bit output sum along with an output carry.



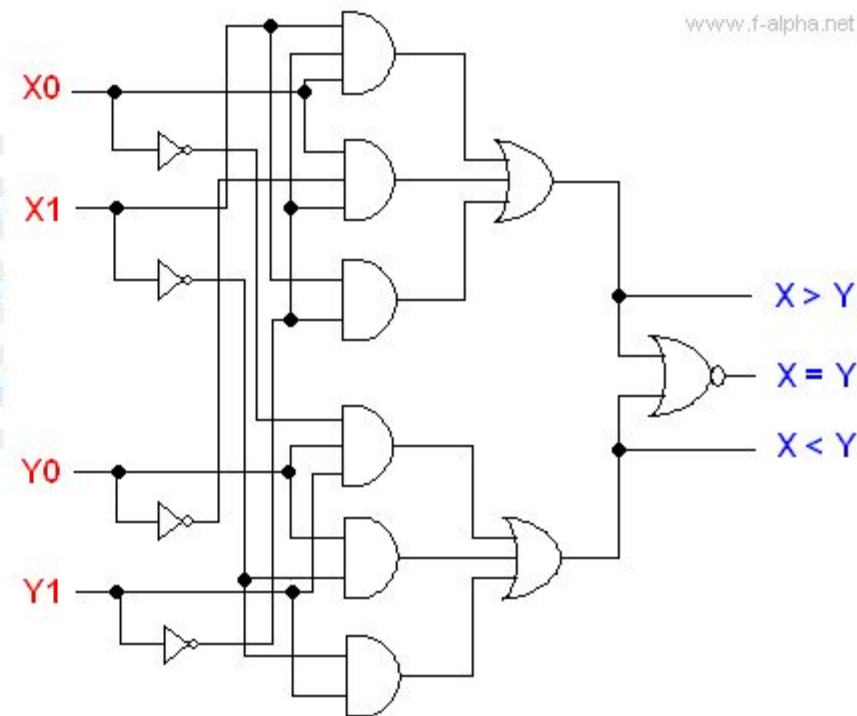
Comparator

- A comparator compares two n-bit binary inputs A and B, generating outputs that indicate whether A is less than, equal to, or greater than B.

1 bit inputs

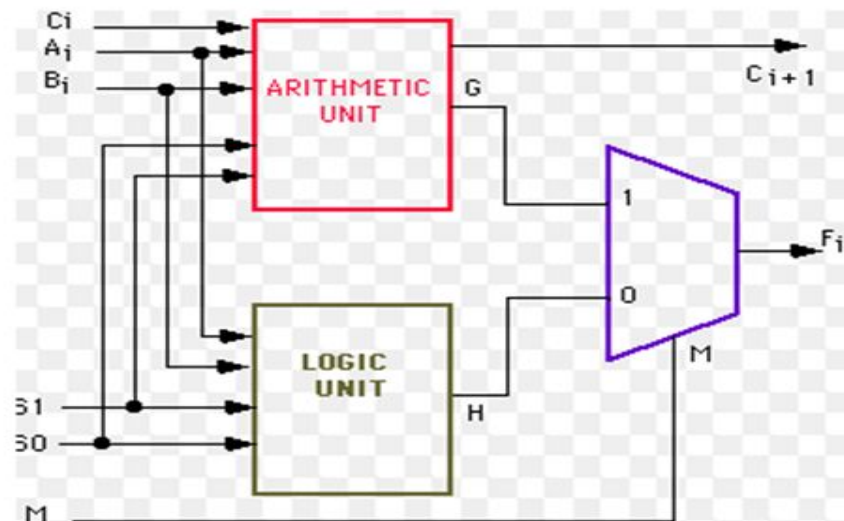
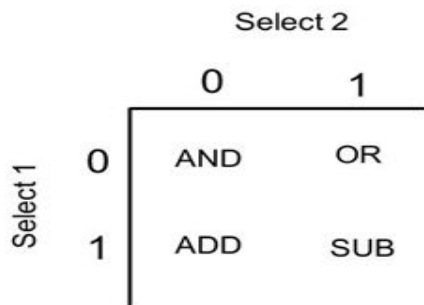


2bit inputs



ALU

- An *ALU* (arithmetic-logic unit) can perform a variety of arithmetic and logic functions on its n -bit inputs A and B .
- The select lines S choose the current function; if there are m possible functions, then there must be at least $\log_2(m)$ select lines.



$O =$ I_0 if $S=0..00$ I_1 if $S=0..01$ $\dots$ I_m if $S=1..11$	$O_0 = 1$ if $I=0..00$ $O_1 = 1$ if $I=0..01$ $\dots$ $O_n = 1$ if $I=1..11$	$sum = A+B$ (first n bits) $carry = (n+1)'$ th bit of $A+B$	$less = 1$ if $A < B$ $equal = 1$ if $A = B$ $greater = 1$ if $A > B$	$O = A \ op \ B$ op determined by S .
	enable input e all O 's 0 if $e=0$	carry-in input C_i $sum = A+B+C_i$		status outputs carry, zero, etc.

Sequential logic design

- A sequential circuit is a digital circuit whose outputs are a function of the current as well as previous input values.
- In other words, sequential logic possesses memory.
- One of the most basic sequential circuits is the flip-flop. A flip-flop stores a single bit.

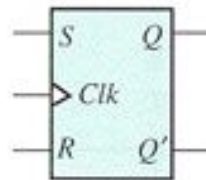
FLIP-FLOP
NAME

FLIP-FLOP
SYMBOL

CHARACTERISTIC
TABLE

CHARACTERISTIC
EQUATION

SR

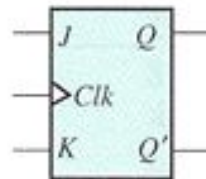


S	R	$Q(next)$
0	0	Q
0	1	0
1	0	1
1	1	NA

$$Q(next) = S + R'Q$$

$$SR = 0$$

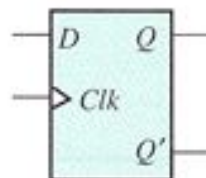
JK



J	K	$Q(next)$
0	0	Q
0	1	0
1	0	1
1	1	Q'

$$Q(next) = JQ' + K'Q$$

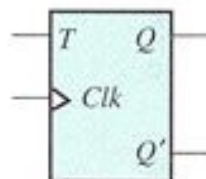
D



D	$Q(next)$
0	0
1	1

$$Q(next) = D$$

T

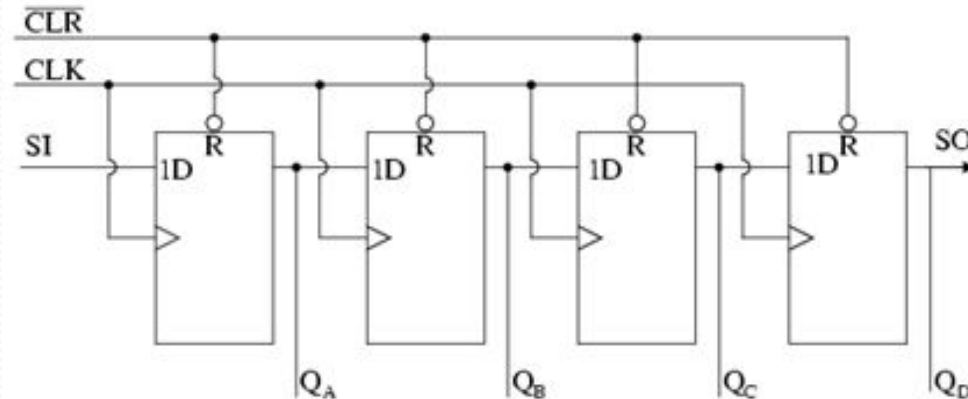


T	$Q(next)$
0	Q
1	Q'

$$Q(next) = TQ' + T'Q$$

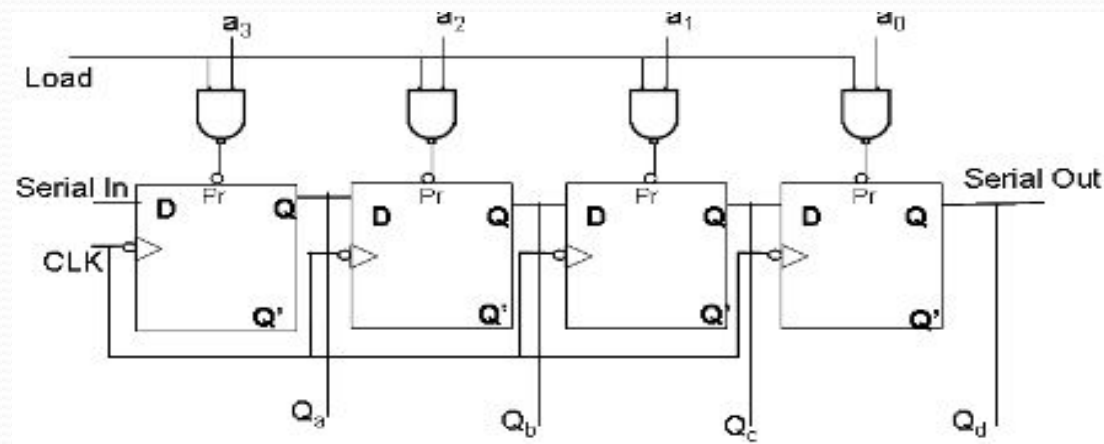
Registers

- A register stores n bits from its n -bit data input I , with those stored bits appearing at its output O .
- A register usually has at least two control inputs, clock and load.
- For a rising-edge-triggered register, the inputs I are only stored when load is 1 and clock is rising from 0 to 1.

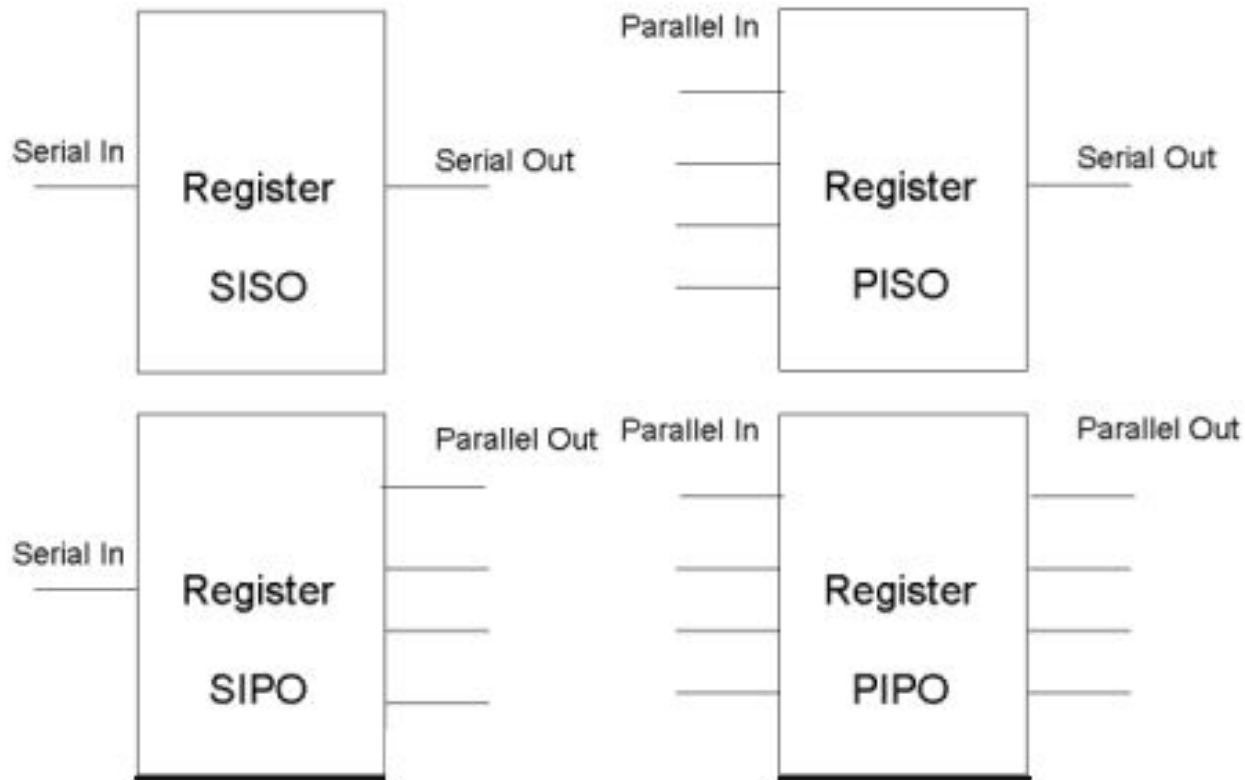


Shift registers

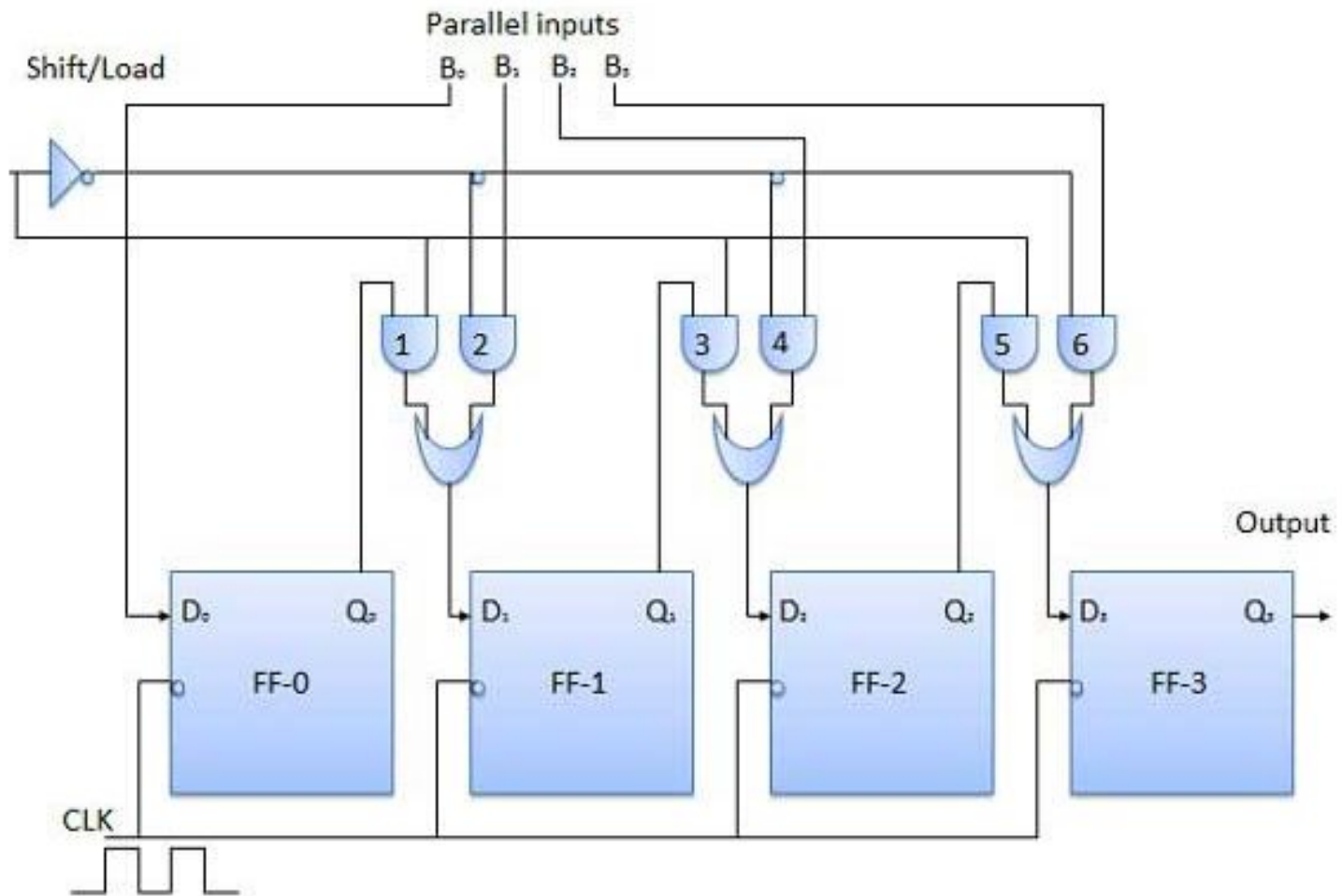
- A shift register has a one-bit data input I , and at least two control inputs clock and shift.
- When clock is rising and shift is 1, the value of I is stored in the (n) 'th bit, while the (n) 'th bit is stored in the $(n-1)$ 'th bit, and likewise, until the second bit is stored in the first bit.
- The first bit is typically shifted out, meaning it appears over an output Q .



Types of SR's



PISO SR

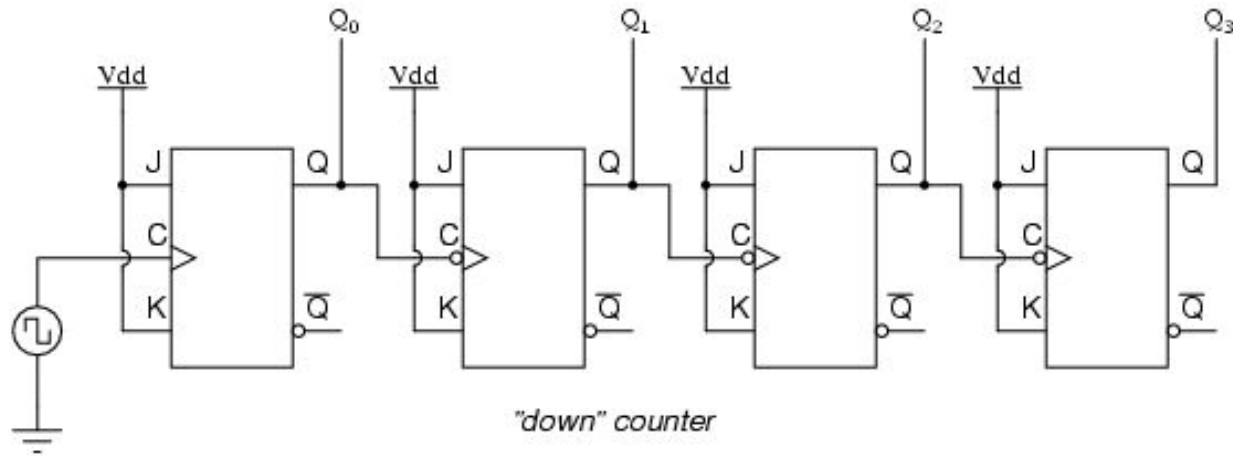


Counters

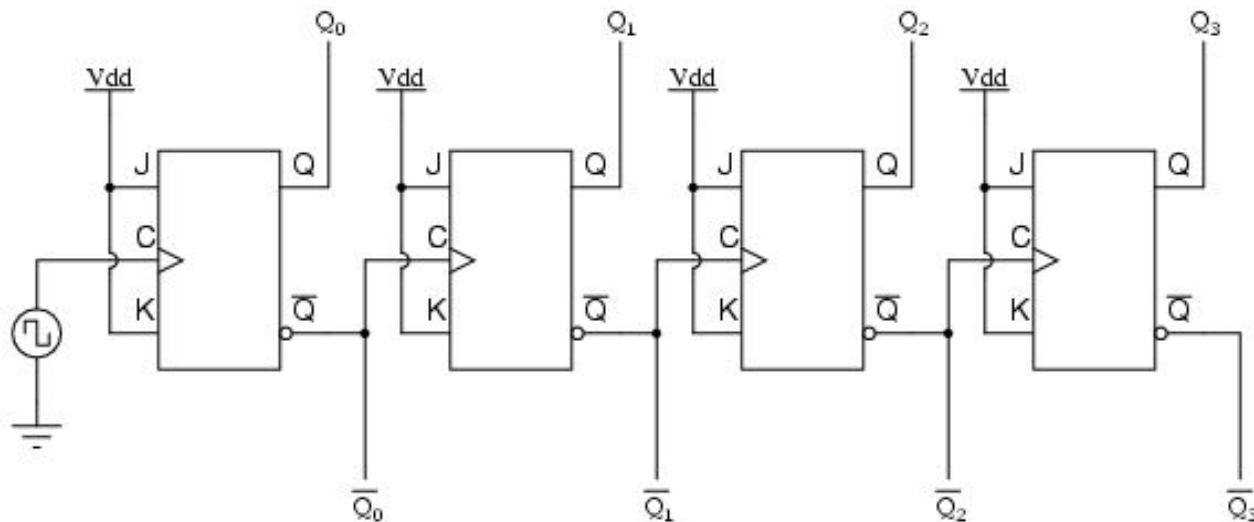
- A counter is a register that can also increment (add binary 1) to its stored binary value.
- A counter has a clear input, which resets all stored bits to 0, and a count input, which enables incrementing on the clock edge.
- There are two types of counters
 - Asynchronous counters(Up/Down) : No need of clock pulse to count
 - Synchronous Counters (Up/Down): need clock pulse to count

4 bit Asynchronous Counter

A four-bit "up" counter

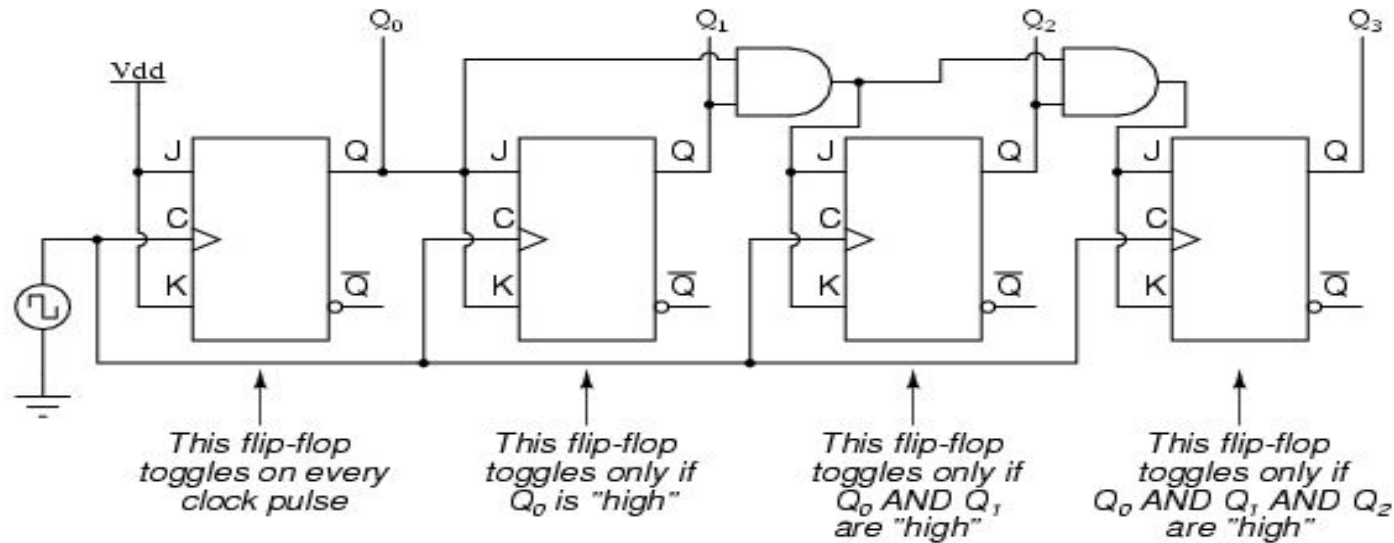


"down" counter

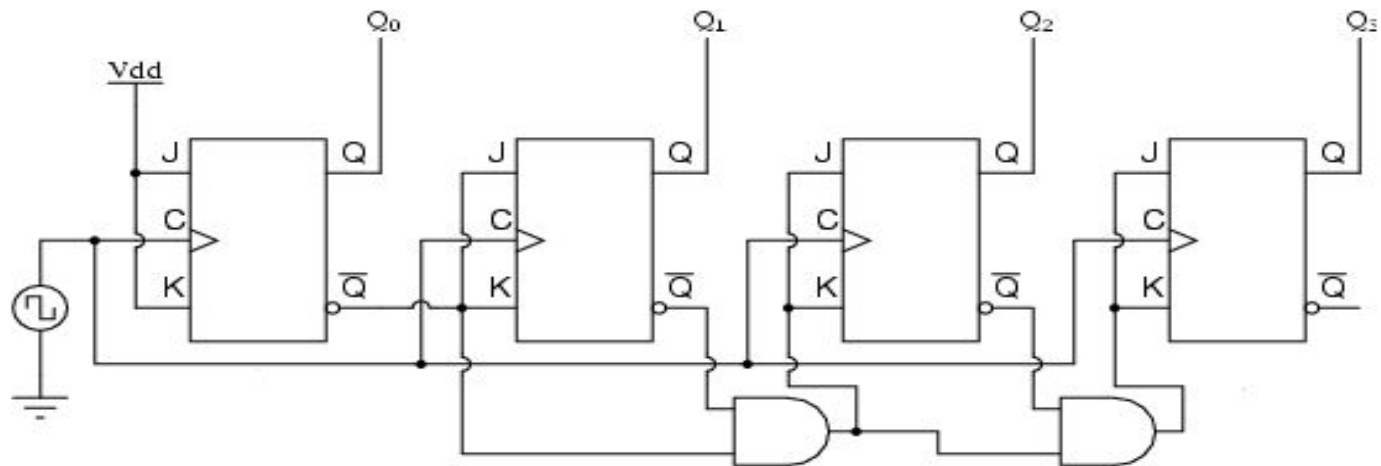


4 bit Synchronous counter

A four-bit synchronous "up" counter

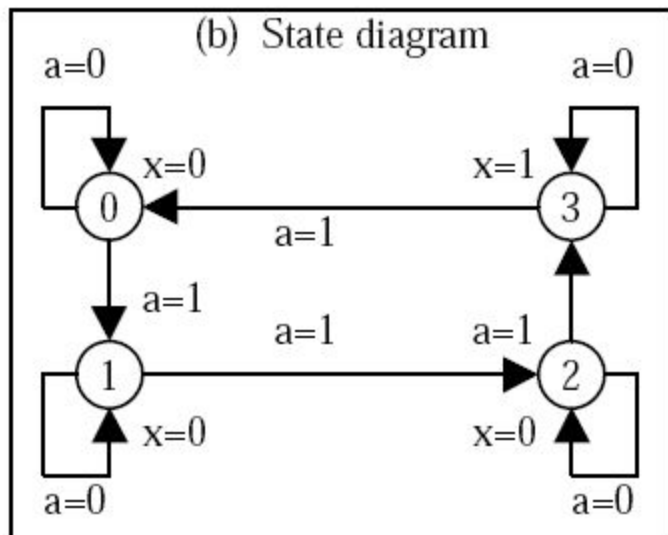


A four-bit synchronous "down" counter



Sequential logic design eg:

- Q: You want to construct a clock divider Slow down your pre-existing clock so that you output a 1 for every four clock cycles.



(d) State table (Moore-type)

Inputs			Outputs		
Q1	Q0	a	I1	I0	x
0	0	0	0	0	0
0	0	1	0	1	
0	1	0	0	1	0
0	1	1	1	0	
1	0	0	1	0	0
1	0	1	1	1	
1	1	0	1	1	1
1	1	1	0	0	

(e) Minimized output equations

		Q1Q0			
		00	01	11	10
I1	a				
	0	0	0	1	1
	1	0	1	0	1

$I1 = Q1'Q0a + Q1a' + Q1Q0'$

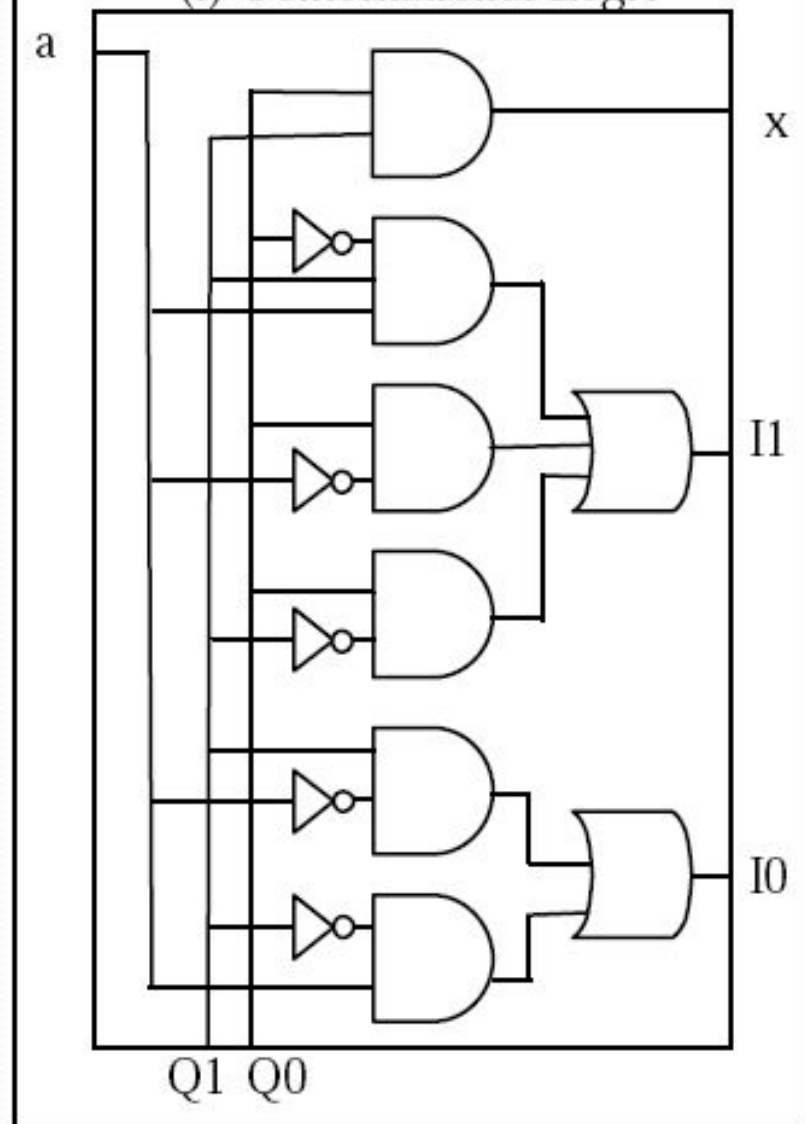
		Q1Q0			
		00	01	11	10
I0	a				
	0	0	1	1	0
	1	1	0	0	1

$I0 = Q0a' + Q0'a$

		Q1Q0			
		00	01	11	10
x	a				
	0	0	0	1	0
	1	0	0	1	0

$x = Q1Q0$

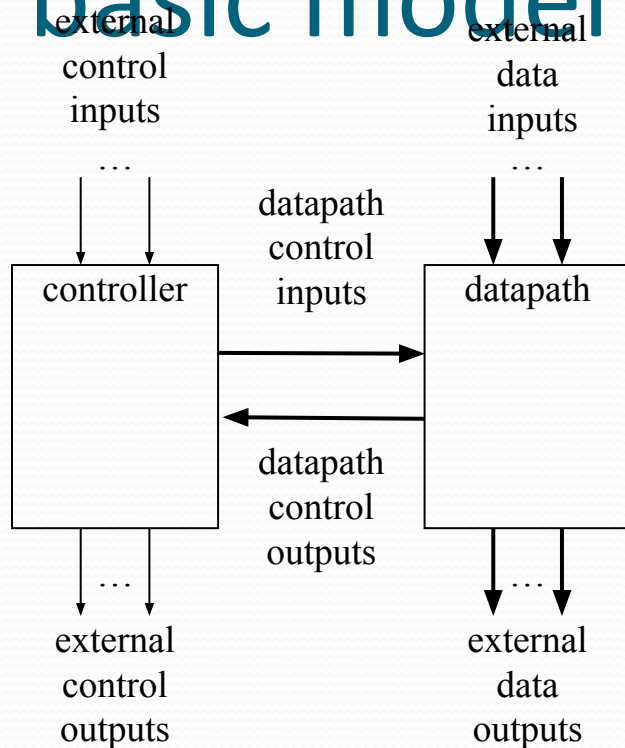
(f) Combinational Logic



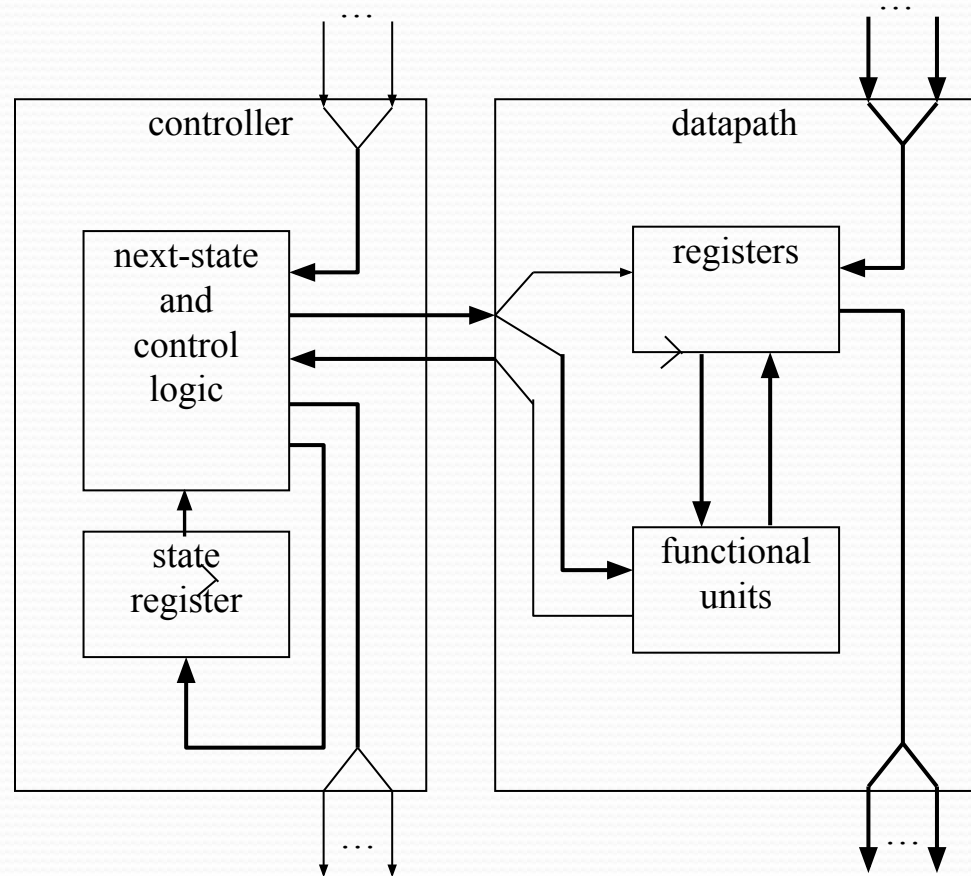


CUSTOM SINGLE-PURPOSE PROCESSOR DESIGN

Custom single-purpose processor basic model



controller and datapath



a view inside the controller and datapath

How?

- Designer can apply the all combinational and sequential logic design techniques to build data-path components and controllers.
- Designer has nearly all the knowledge ,he needs to build a custom single-purpose processor for a given program, since a processor consists of a controller and a data-path.
- Here it describe a technique for building such a processor.

Explanation with eg;

- QSTN: Design a CSP circuit to find greatest common divisor (GCD) of two no's, ie; if the inputs are 12 and 8, the output should be 4 or If the inputs are 13 and 5, the output should be 1.

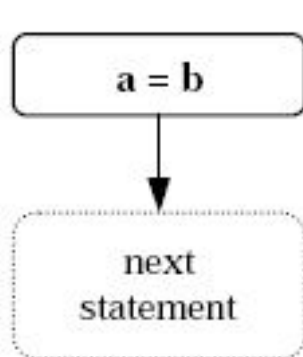
Solution

- To begin building our single-purpose processor implementing the GCD program, we first convert our program into a complex state diagram called finite state machine with data (FSMD) .
- In which states and arcs may include arithmetic expressions, and these expressions may use external inputs and outputs or variables.
- First we have to learn how “while loop” and “ if- else” statement can be convert to state diagram.

Templates for creating a state diagram from a program.

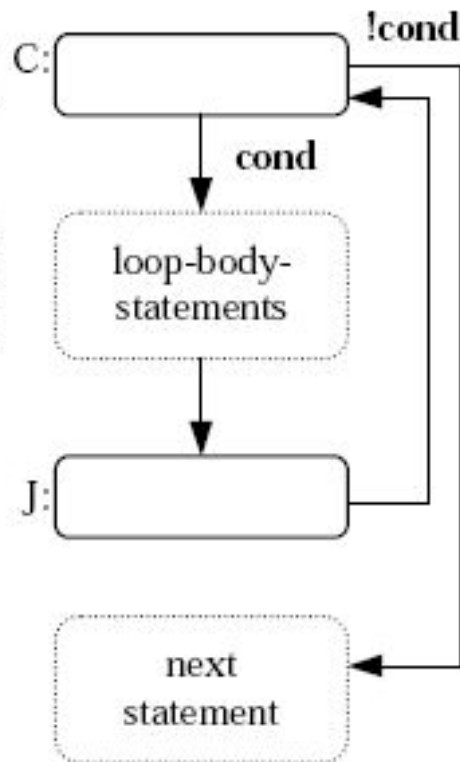
Assignment
statement

a = b
next statment



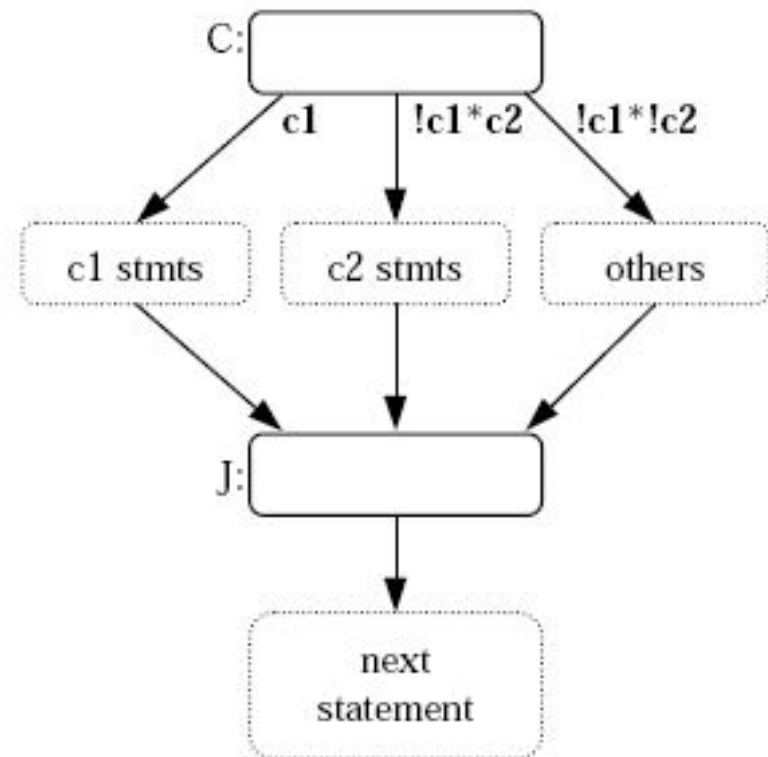
Loop
statement

while (cond) {
loop-body-
statements
}
next statement



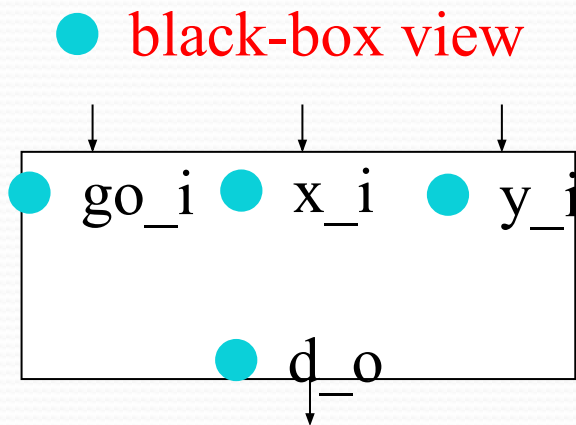
Branch
statement

if (c1)
c1 stmts
else if c2
c2 stmts
else
other stmts
next statement



Step1: Problem view with functionality

desired functionality,



```
0: Int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
9:   }
9:   d_o = x;
}
```

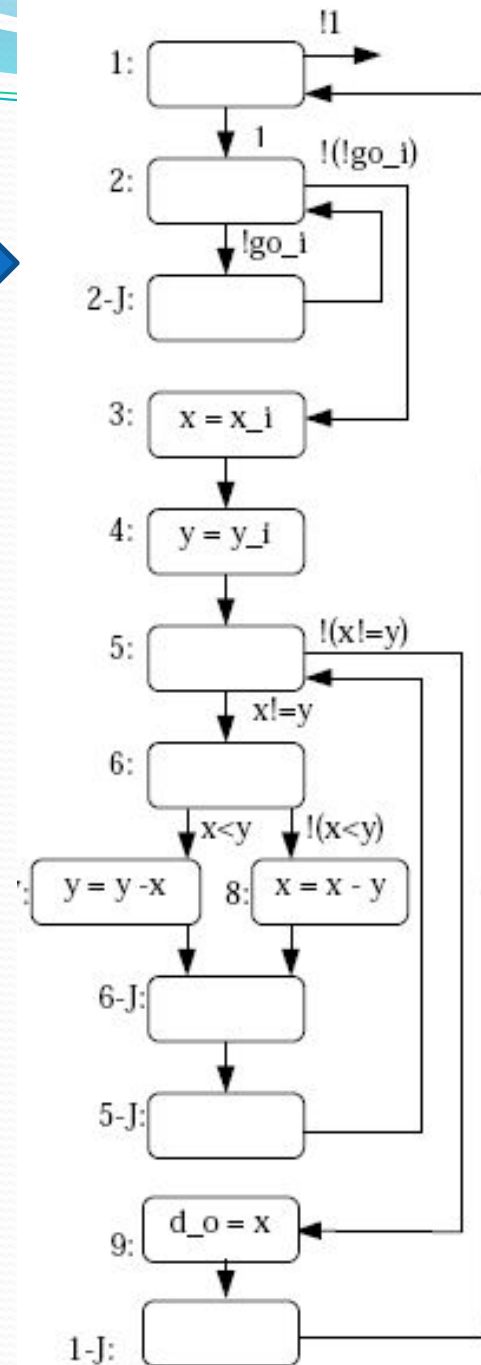
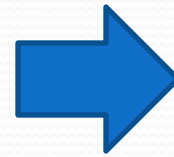
We can use templates to convert this program to a state diagram.

Step 2: The state diagram

```

0: Int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
9:   d_o = x;
1-J:

```

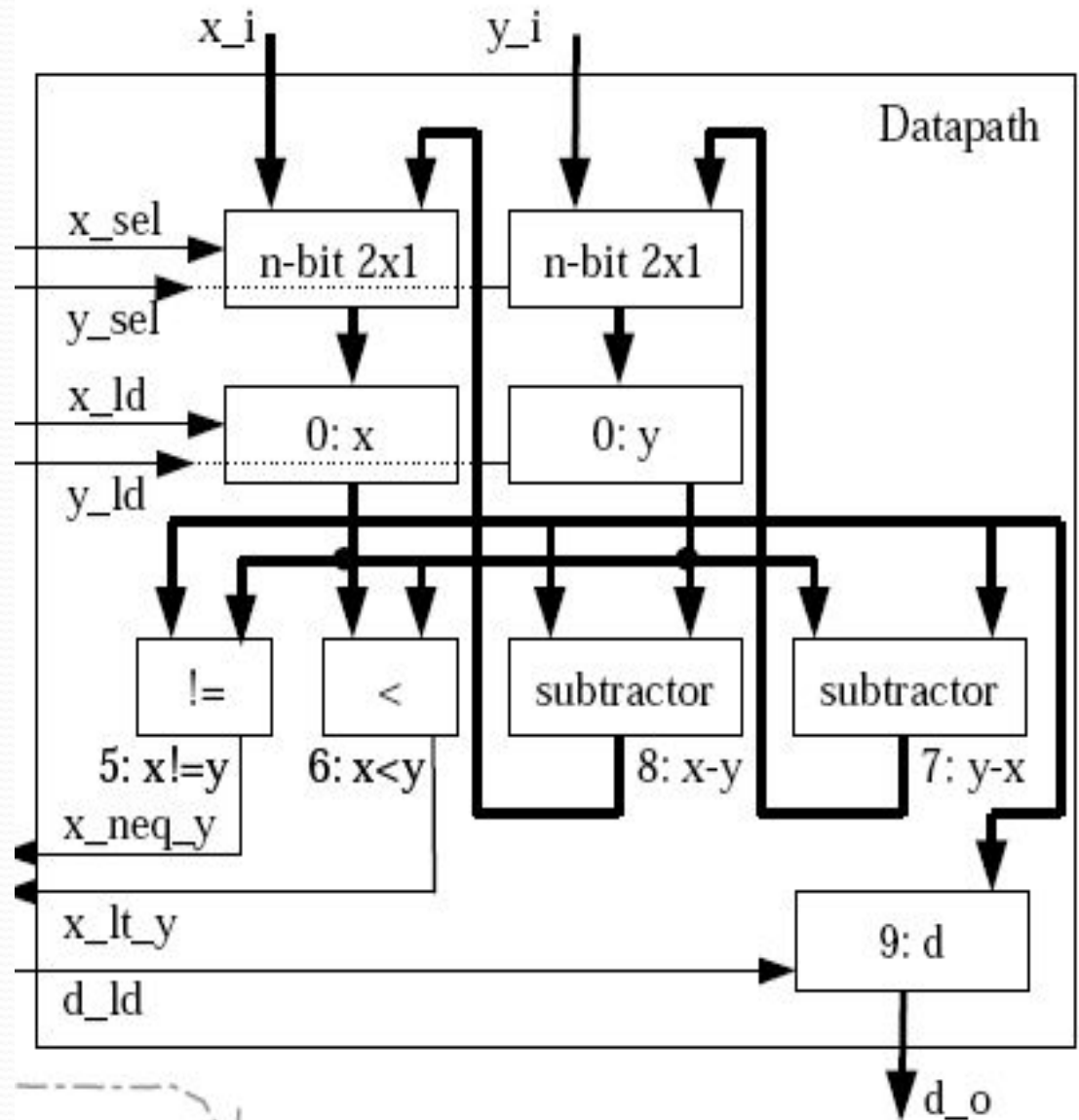
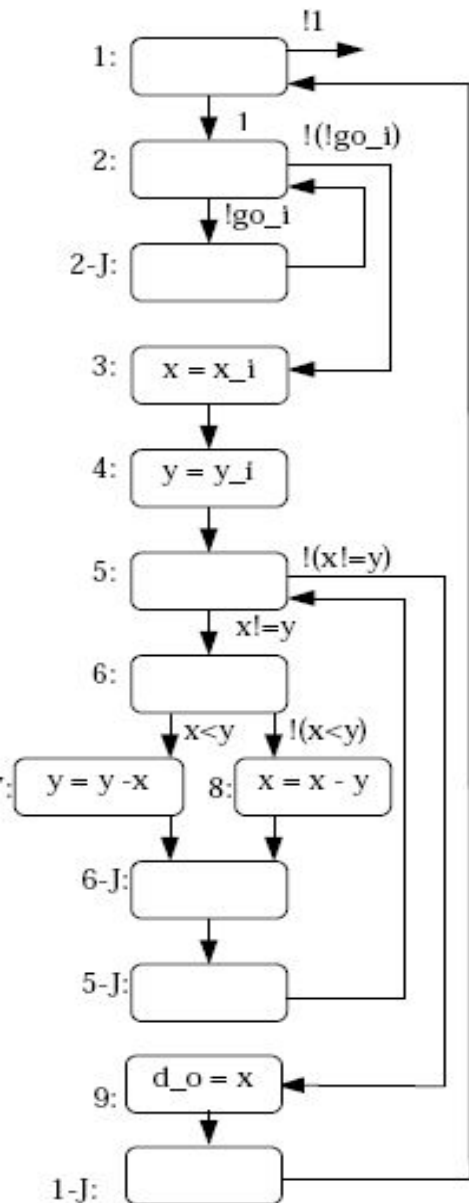


Step 3: Divide the functionality into a datapath part and a controller part

- The data path part should consist of an interconnection of combinational and sequential components.
- The controller part should consist of a basic state diagram, i.e., one containing only boolean actions and conditions.

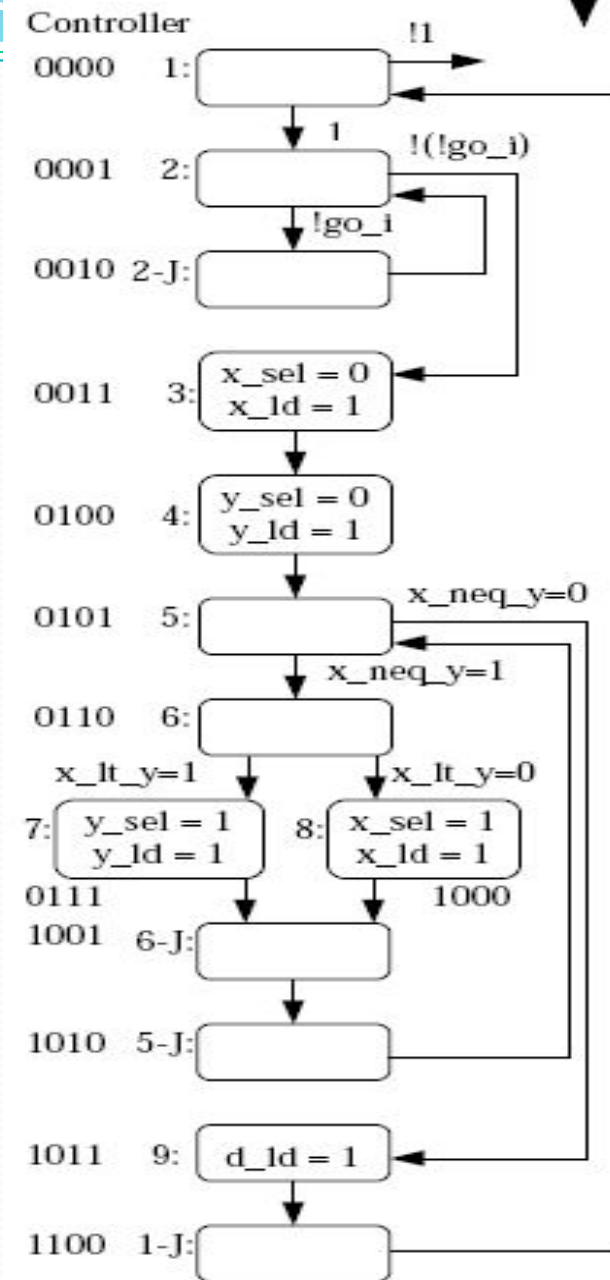
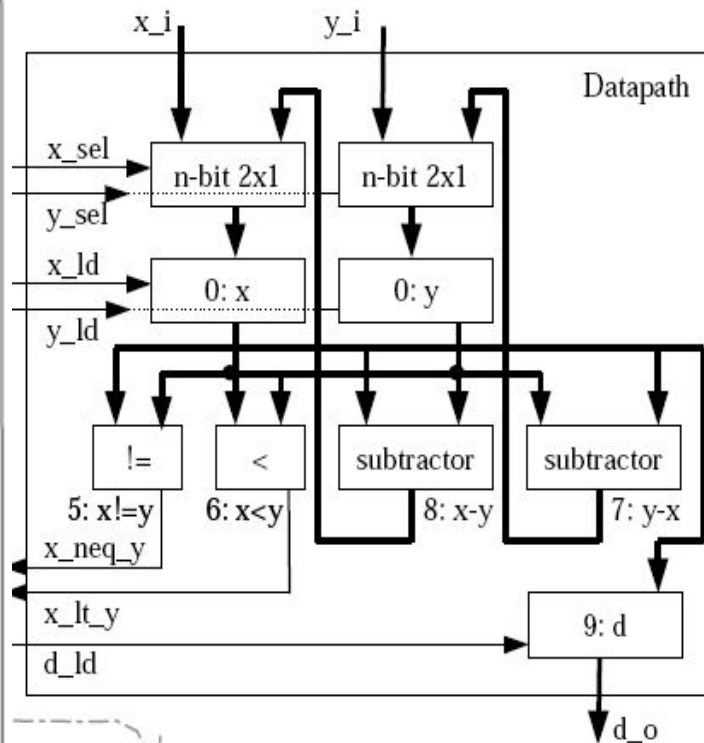
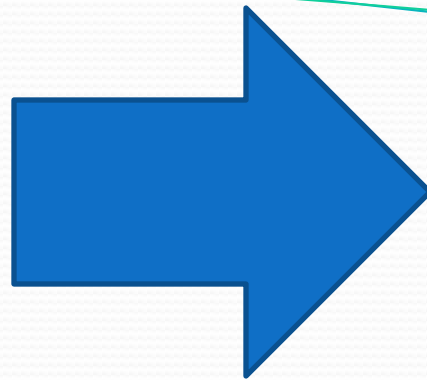
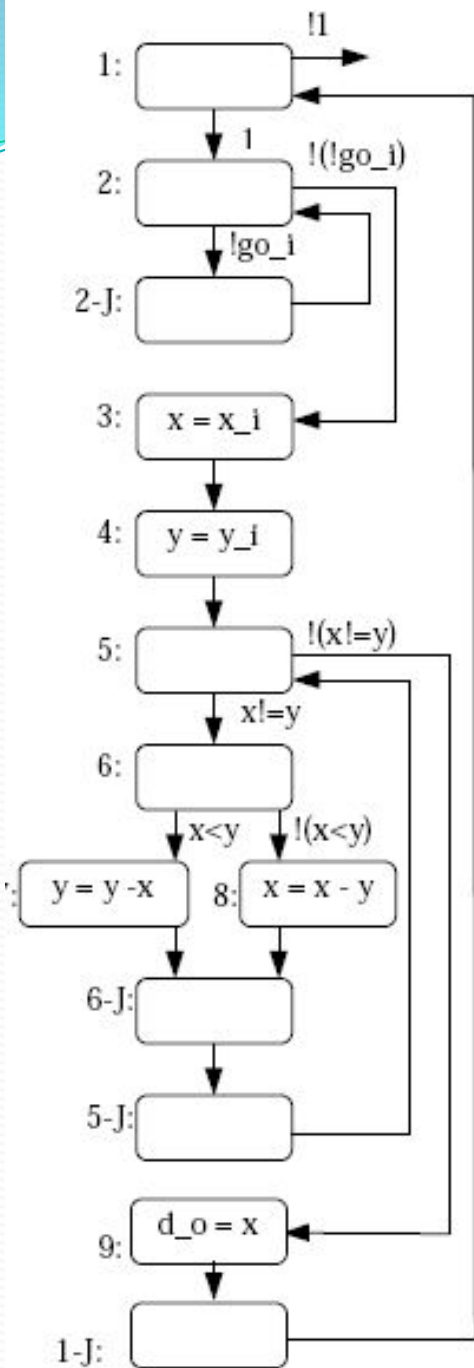
- Construction of datapath through 4 steps:
 - 1. we create a register for any declared variable. In the example, these are x and y. We treat an output port as having an implicit variable, so we create a register d and connect it to the output port. We also draw the input and output ports.
 - 2. Second, we create a functional unit for each arithmetic operation in the state diagram. In the example, there are two subtractions, one comparison for less than, and one comparison for inequality, yielding two subtractors and two comparators, as shown in the figure.
 - 3. Third, we connect the ports, registers and functional units. For each write to a variable in the state diagram, we draw a connection from the write's source (an input port, a functional unit, or another register) to the variable's register. For each arithmetic and logical operation, we connect sources to an input of the operation's corresponding functional unit. When more than one source is connected to a register, we add an appropriately-sized multiplexor.
 - 4. Finally, we create a unique identifier for each control input and output of the datapath components.

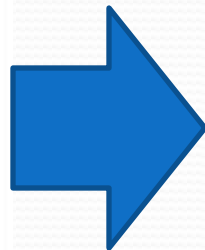
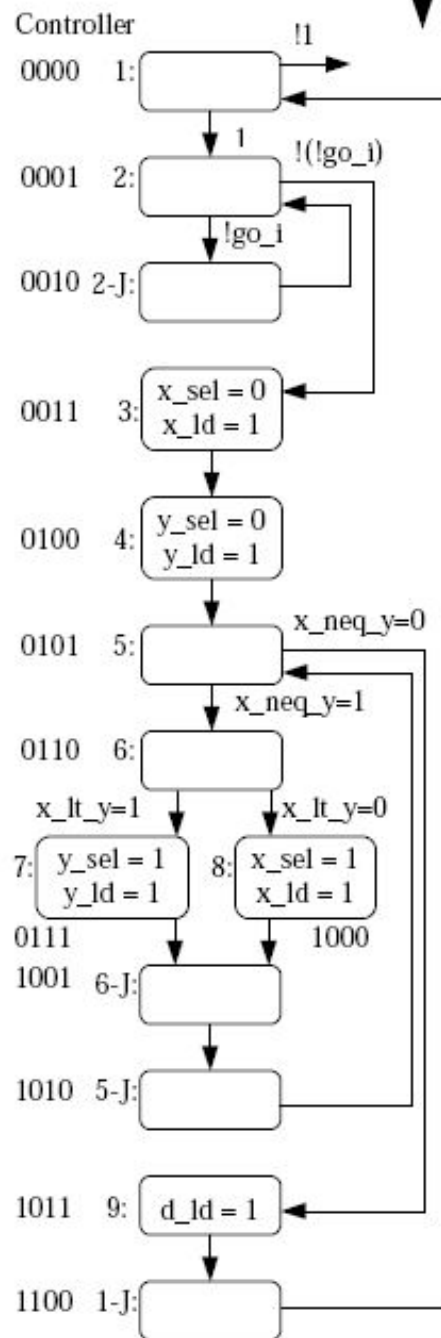
The datapath



● Construction of controller part

- We replace every variable write by actions that set the select signals of the multiplexor in front of the variable's register's such that the write's source passes through, and we assert the load signal of that register.
- We replace every logical operation in a condition by the corresponding functional unit control output.



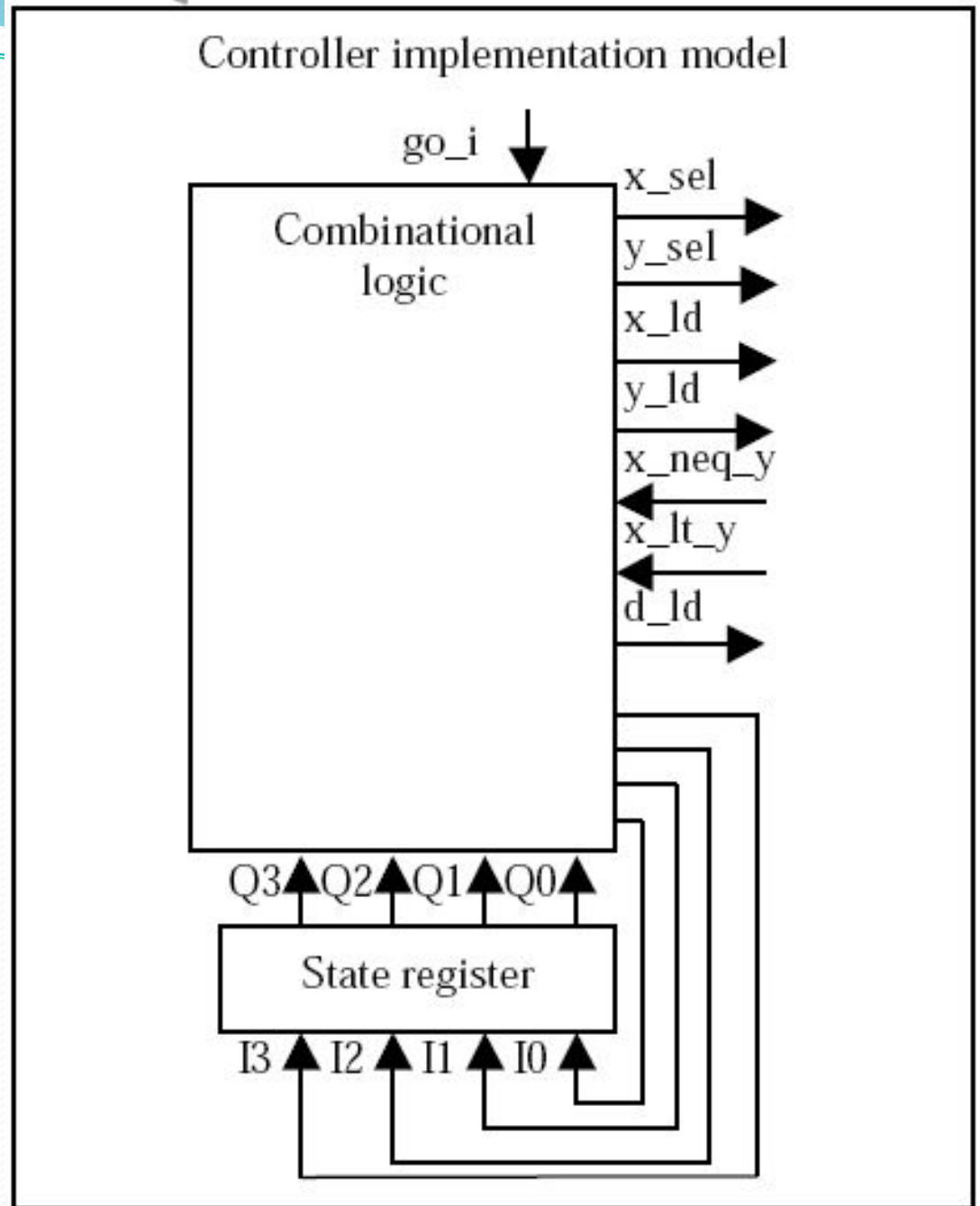
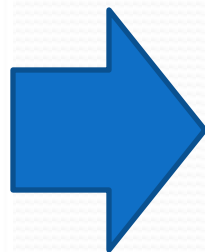
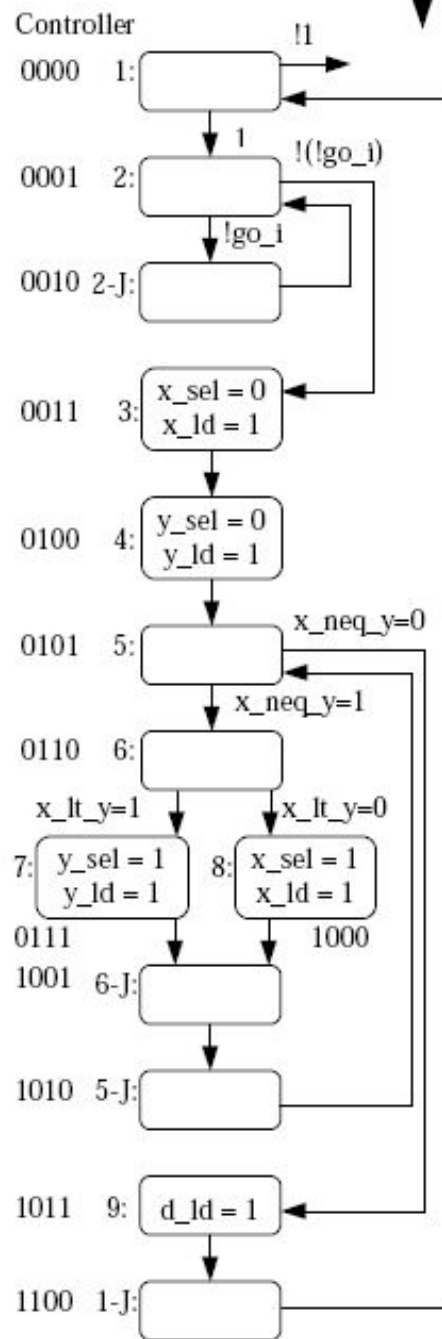


State table for the GCD example.

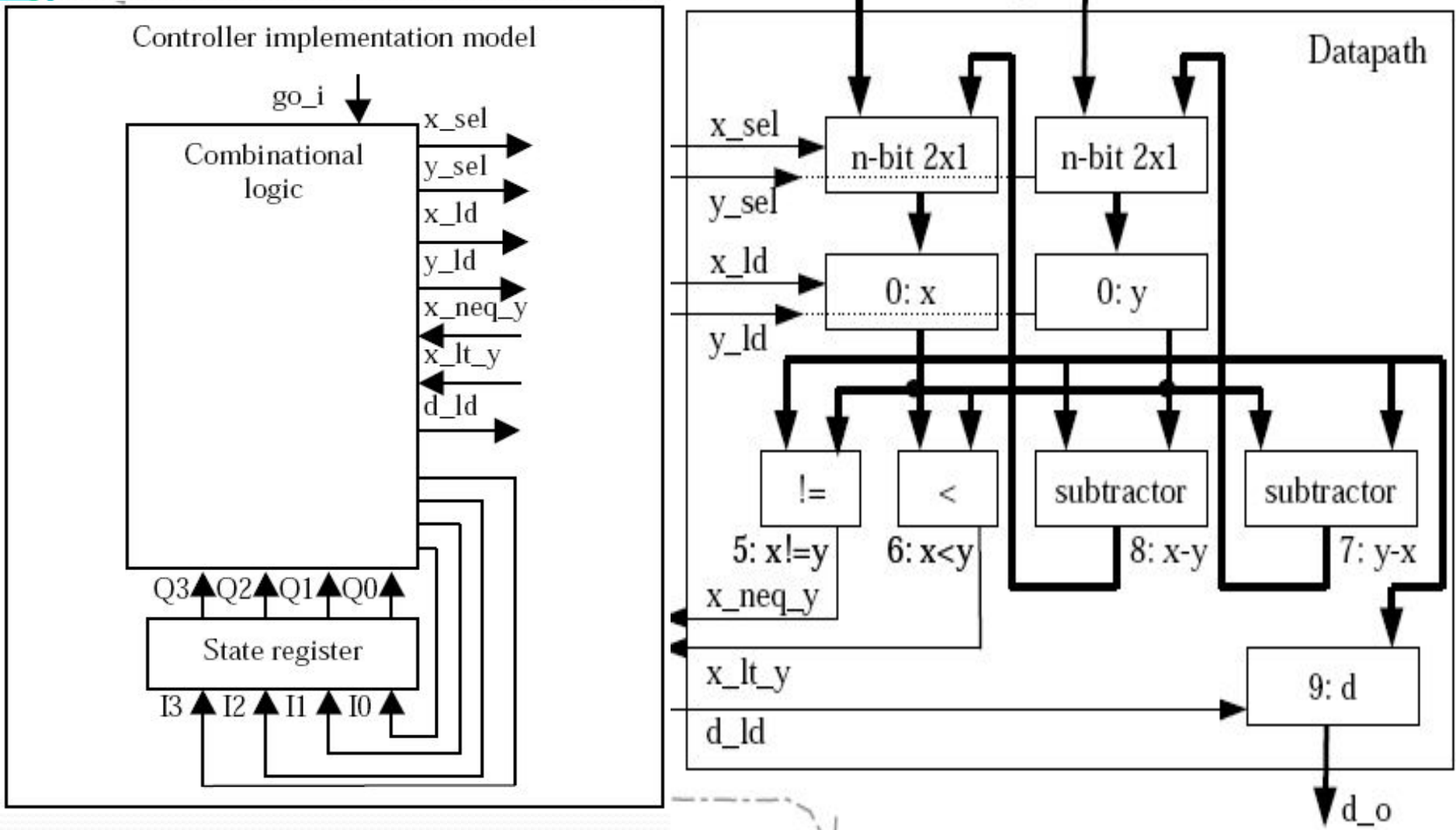
Inputs							Outputs								
Q3	Q2	Q1	Q0	x_neq_y	x_lt_y	go_i	I3	I2	I1	I0	x_sel	y_sel	x_ld	y_ld	d_ld
0	0	0	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	0	1	*	*	0	0	0	1	0	X	X	0	0	0
0	0	0	1	*	*	1	0	0	1	1	X	X	0	0	0
0	0	1	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	1	1	*	*	*	0	1	0	0	0	X	1	0	0
0	1	0	0	*	*	*	0	1	0	1	X	0	0	1	0
0	1	0	1	0	*	*	1	0	1	1	X	X	0	0	0
0	1	0	1	1	*	*	0	1	1	0	X	X	0	0	0
0	1	1	0	*	0	*	1	0	0	0	X	X	0	0	0
0	1	1	0	*	1	*	0	1	1	1	X	X	0	0	0
0	1	1	1	*	*	*	1	0	0	1	X	1	0	1	0
1	0	0	0	*	*	*	1	0	0	1	1	X	1	0	0
1	0	0	1	*	*	*	1	0	1	0	X	X	0	0	0
1	0	1	0	*	*	*	0	1	0	1	X	X	0	0	0
1	0	1	1	*	*	*	1	1	0	0	X	X	0	0	1
1	1	0	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	0	1	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	1	*	*	*	0	0	0	0	X	X	0	0	0

* - indicates all possible combinations of 0's and 1's

X - indicates don't cares

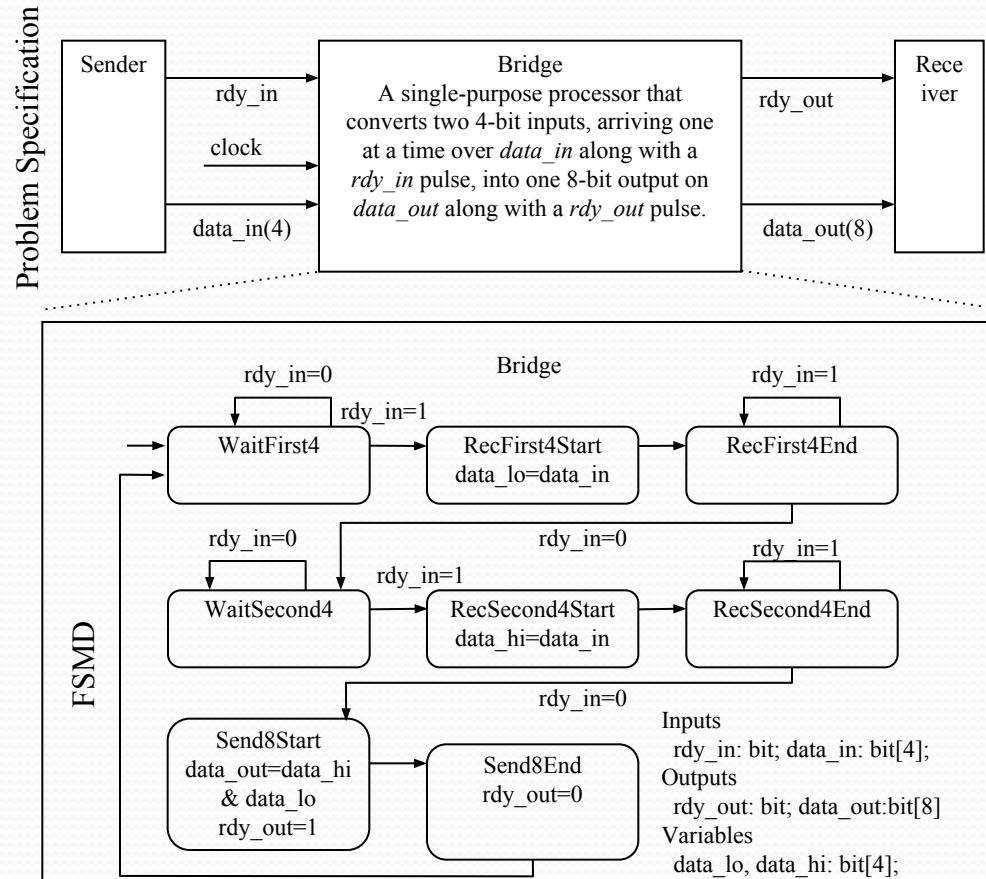


Data path and controller for GCD

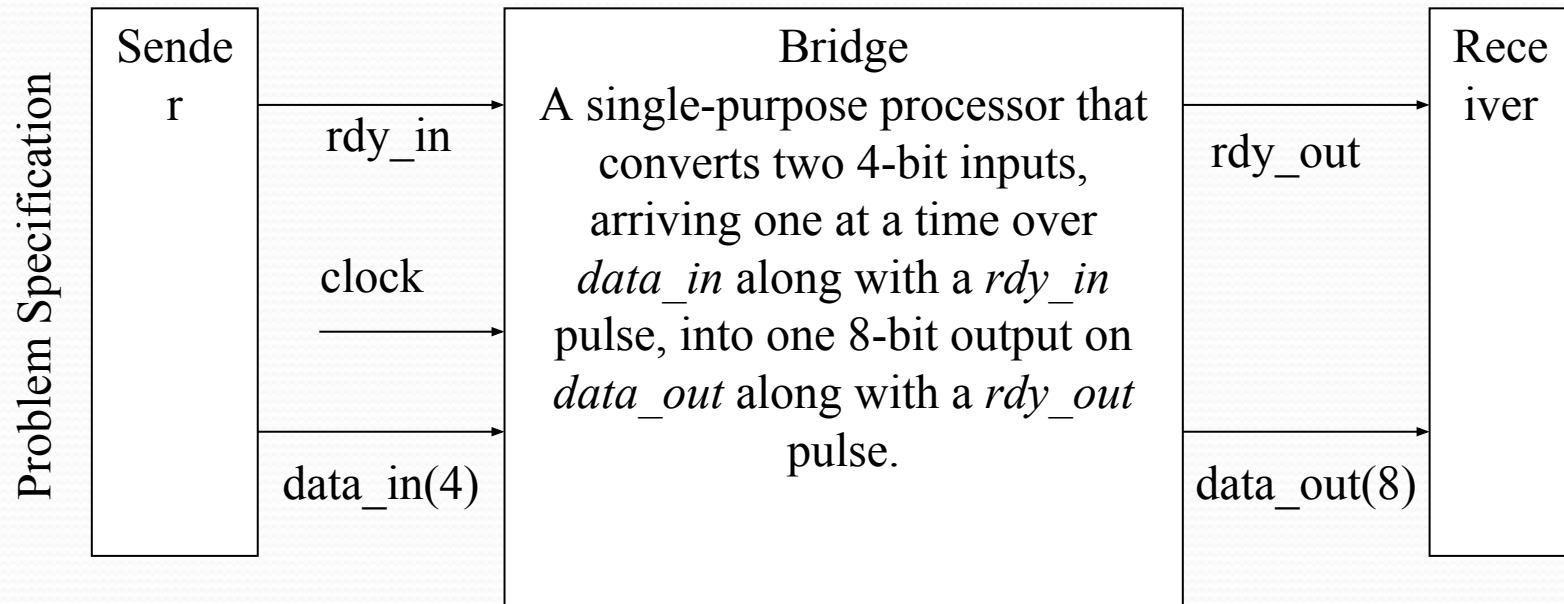


RT-level custom single-purpose processor design

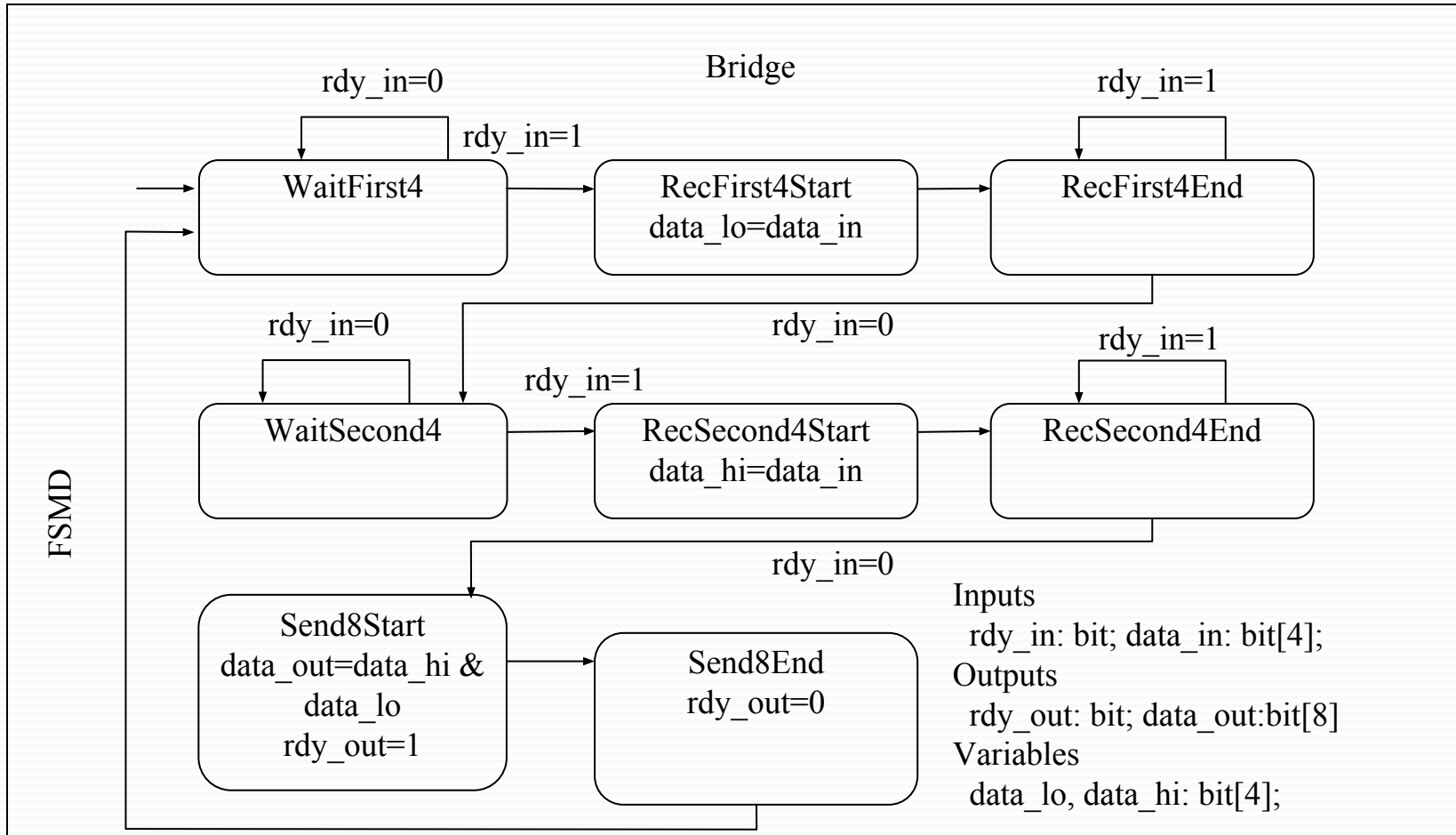
- We often start with a state machine
 - Rather than algorithm
 - Cycle timing often too central to functionality
- Example
 - Bus bridge that converts 4-bit bus to 8-bit bus
 - Start with FSMD
 - Known as register-transfer (RT) level
 - Exercise: complete the design



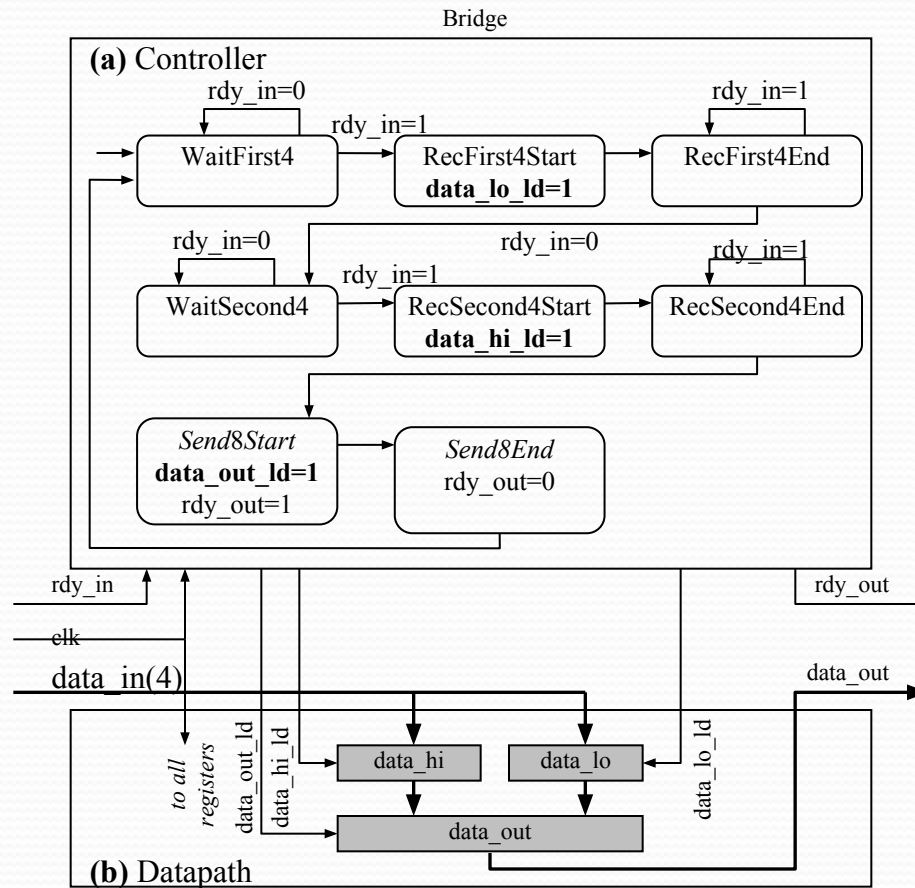
Problem Specification



FSMD for the Problem



RT-level custom single-purpose processor design (cont')



Optimizing Custom single-purpose processors design

- Optimization is the task of making design metric values the best possible
- Optimization in CSPP design means,
 - Optimizing the original program
 - Optimizing the FSMD
 - Optimizing the datapath
 - Optimizing the FSM

Optimizing the original program

- Analyze program attributes and look for areas of possible improvement
 - number of computations
 - size of variable
 - time and space complexity
 - operations used
 - multiplication and division very expensive

GCD program

original

program

```
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
10:  }
11:  d_o = x;
12: }
```

replace the subtraction
operation(s) with
modulo operation in
order to speed up
program

optimized program

```
0: int x, y, r;
1: while (1) {
2:   while (!go_i);
3:   // x must be the larger number
4:   if (x_i >= y_i) {
5:     x=x_i;
6:     y=y_i;
7:   }
8:   else {
9:     x=y_i;
10:    y=x_i;
11:  }
12:  while (y != 0) {
13:    r = x % y;
14:    x = y;
15:    y = r;
16:  }
17:  d_o = x;
18: }
```

GCD(42, 8) - 9 iterations to complete the loop
x and y values evaluated as follows : (42, 8),
(43, 8), (26,8), (18,8), (10, 8), (2,8), (2,6),
(2,4), (2,2).

GCD(42,8) - 3 iterations to complete the loop
x and y values evaluated as follows: (42, 8),
(8,2), (2,0)

Optimizing the Finite state machine with datapath

- Areas of possible improvements

- merge states

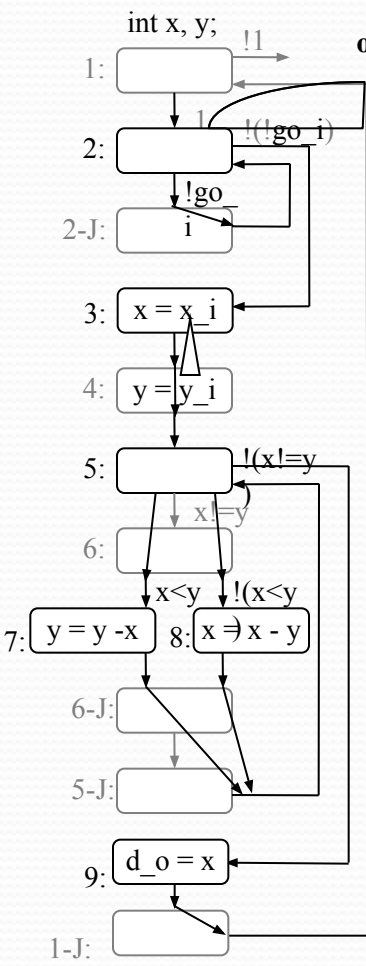
- states with constants on transitions can be eliminated, transition taken is already known
 - states with independent operations can be merged

- separate states

- states which require complex operations ($a*b*c*d$) can be broken into smaller states to reduce hardware size

- Scheduling

- Scheduling the task of assigning operations from the original program to states in an FSMD



original FSM

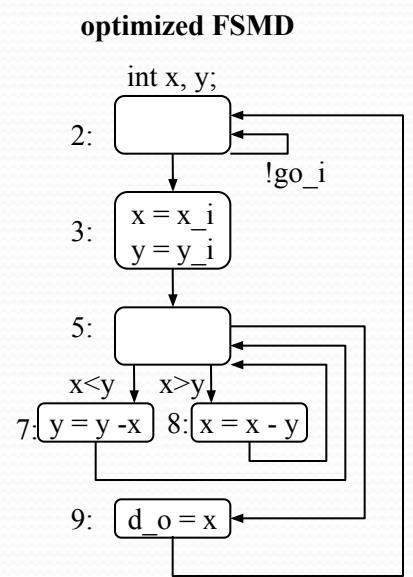
eliminate state 1 – transitions have constant values

merge state 2 and state 2J – no loop operation in between them

merge state 3 and state 4 – assignment operations are independent of one another

merge state 5 and state 6 – transitions from state 6 can be done in state 5
eliminate state 5J and 6J –

transitions from each state can be done from state 7 and state 8, respectively
eliminate state 1-J – transition from state 1-J can be done directly from state 9



optimized FSM

Optimizing the datapath

- Sharing of functional units
 - one-to-one mapping, as done previously, is not necessary
 - if same operation occurs in different states, they can share a single functional unit
- Multi-functional units
 - ALUs support a variety of operations, it can be shared among operations occurring in different states

Optimizing the FSM

- State encoding

- task of assigning a unique bit pattern to each state in an FSM
- size of state register and combinational logic vary
- can be treated as an ordering problem

- State minimization

- task of merging equivalent states into a single state
 - state equivalent if for all possible input combinations the two states generate the same outputs and transitions to the next same state

Introduction to Application

Specific Integrated Circuit (ASIC)

- In our day to day life, we come across various types of electronic gadgets. One of the technologies that brought forth a revolution in the production of electronics is “Integrated Circuit“. This technology reduced the size of electronic products by increasing the density of logic gates per chip. Today we have different types and configurations of IC’s. As we observe around we find that some ICs can be used only for one specific application while some ICs can be reprogrammed and used for various applications. These types of ICs are named as ASICs. But how do they differ? How is it possible to reprogram them? Why some ICs can’t be reprogrammed?
- ASIC full form is Application Specific Integrated Circuit. These circuits are application specific .i.e. tailored made ICs for a particular application. These are usually designed from root level based on the requirement of the particular application. Some of the basic application-specific integrated circuit examples are chips used in toys, the chip used for interfacing of memory and microprocessor etc...These chips can be used only for that one application for which these are designed. Presumably, these types of ICs are preferred only for those products which have a large production run. As ASICs are designed from the root level they have high cost and are recommended only for high volume productions.

Conti...

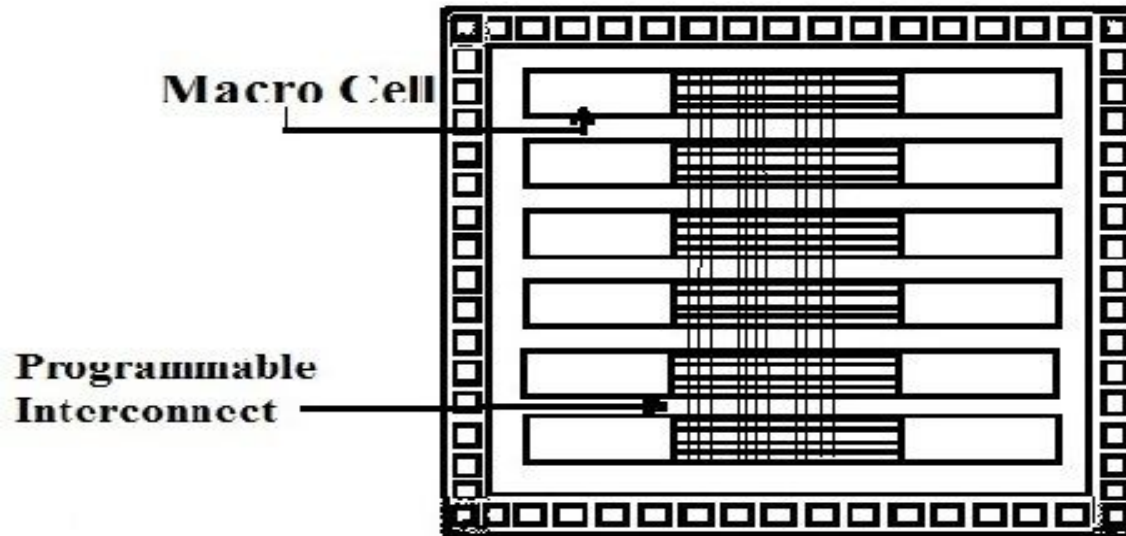
- The main advantage of ASIC is reduced chip size as a large number of functional units of a circuit are constructed over a single chip. Modern ASIC generally includes a 32-bit **microprocessor**, memory blocks, network circuits etc...Such type of ASICs is known as **System on Chip**.

Programmable ASIC

There are two types of programmable ASICs. They are PLD and FPGA

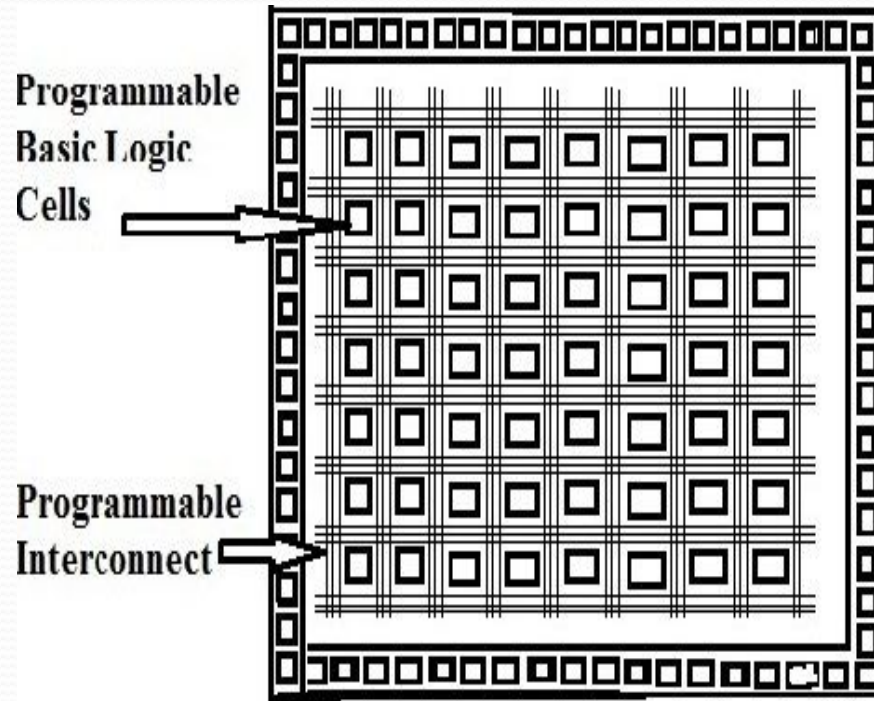
- **PLDs (Programmable Logic Devices)**
- These are the standard cells readily available. We can program a PLD to customized a part of the application, so they are considered as ASIC. We can use different methods and software to program a PLD. These contain a regular matrix of logic cells usually programmable array logic along with flip-flops or latches. Here interconnects are present as a single large block. PROM is a common example of this IC. EPROM uses MOS transistors as interconnect so by applying high voltage we can program it. PLDs have no customized logic cells or interconnect. These have a fast design turnaround.

Conti...



FPGA

Where PLDs have programmable array logic as logic cells **FPGA** has gate array-like arrangement. PLDs are smaller and less complex than FPGAs. Due to its flexibility and characteristics, FPGA is replacing **TTL** in microelectronic systems. Design turnaround is only a few hours •



Conti....

- The core consists of programmable basic logic cells which can perform both **combinational and sequential logic**. We can program logic cells and interconnect using some methods. Basic logic cells are surrounded by the matrix of programmable interconnects and the core is surrounded by programmable I/O cells.
- FPGA usually comprises of configurable logic blocks, configurable I/O blocks, programmable interconnects, clock circuitry, ALU, memory, decoders.

Examples of ASIC

- Having known the different characteristics of ASIC now let us see some examples of ASIC.
- - Standard cell-based ASIC:** LCB 300k, 500k from LSI Logic Company, SIG1, 2, 3 families from ABB Hafo Inc., GCS90K of GCS Plessey.
 - Gate Array Products:** AUA20K from Harris Semiconductor, SCX6Bxx from National Semiconductors, TGC/TEC families from Texas Instruments.
 - PLD Products:** PAL family of Advanced Micro Devices, GAL family from Philips Semiconductors, XC7300 and EPLD from XILINX.
 - FPGA Products:** XC2000, XC3000, XC4000, XC5000 series from XILINX, pASIC1 of QuickLogic, MAX5000 from Altera.

Applications of ASIC

- The uniqueness of ASIC has revolutionized the way electronics are manufactured. These reduced the die sizes while increasing the density of **logic gates** per chip. ASICs are usually preferred for high-level applications. ASIC chip is used as IP cores for satellites, ROM manufacturing, **Microcontroller** and various types of applications in the medical and research sectors. One of the trending applications of ASIC is BITCOIN MINER.
- Mining of cryptocurrency requires larger power and high-speed hardware. A general purpose CPU cannot provide such a higher computing capacity at high speed. ASIC bitcoin miners are chips built into specially designed motherboards and **power supplies**, constructed into a single unit. It is a purposely designed hardware right down to the chip level for bitcoin mining. These units can execute the algorithm of only single cryptocurrency. For a different type of cryptocurrency presumably, we require another miner.